Advanced Artificial Intelligence and Machine Learning

Transformer Architecture Variants: BERT & T5



Learning goals

- Autoregressive vs. masked language modeling, and how to measure LM performance
- BERT: architecture, pre-training (MLM, NSP), and fine-tuning
- Transfer learning and self-supervision
- BERTology and the BERT-based family: RoBERTa, ALBERT, DistilBERT
- Post-BERT objectives (ELECTRA, XLNet) and T5's text-to-text framing

ARLMs vs. MLM

AGAIN: WHAT IS A LANGUAGE MODEL?

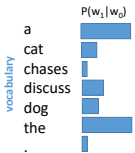
- Statistical model that predicts text that fits well for a given context (typically also text)
- Auto-regressive LMs (ARLMs)
  - Predict one word that is highly likely given a prompt (previous words)
  - For predicting an entire text, repeat the process (i.e., extend the prompt with previously predicted words)
  - To predict a text from scratch, use an extra symbol `<START>` as the initial prompt

ARLM: TOY EXAMPLE

<START>

w_0

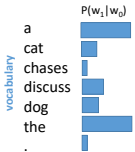
ARLM: TOY EXAMPLE



<START>

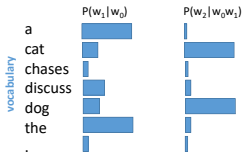
$\mathbf{w_0}$

ARLM: TOY EXAMPLE



<START>	the
w_0	w_1

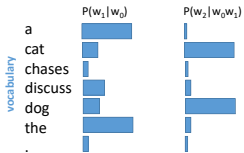
ARLM: TOY EXAMPLE



<START> the

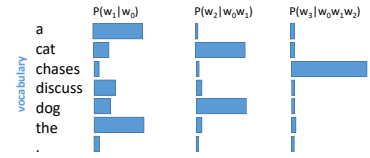
w_0 w_1

ARLM: TOY EXAMPLE



<START>	the	dog
w_0	w_1	w_2

ARLM: TOY EXAMPLE

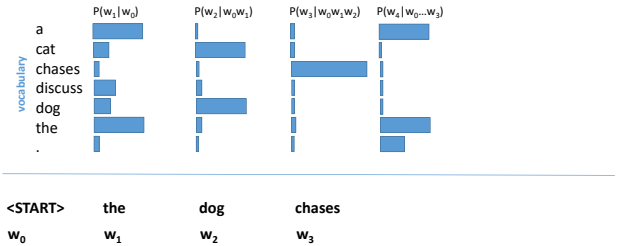


<START>	the	dog
w_0	w_1	w_2

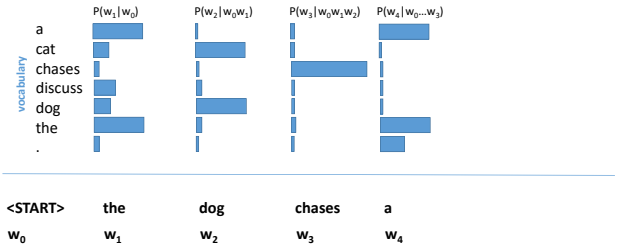
ARLM: TOY EXAMPLE



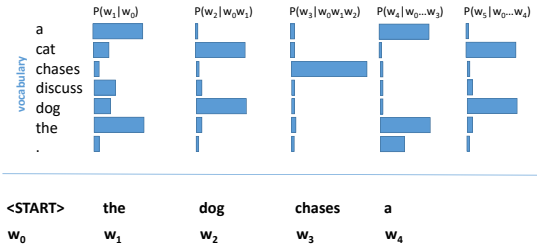
ARLM: TOY EXAMPLE



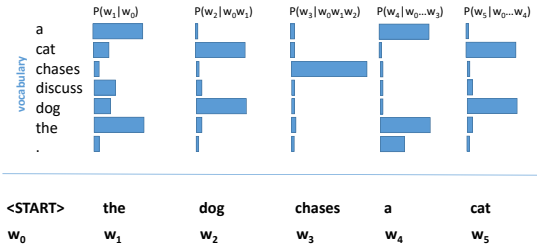
ARLM: TOY EXAMPLE



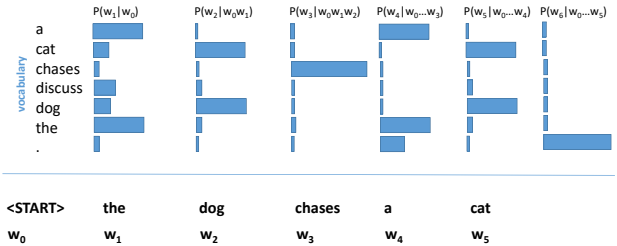
ARLM: TOY EXAMPLE



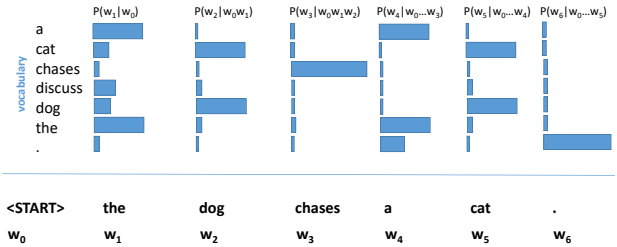
ARLM: TOY EXAMPLE



ARLM: TOY EXAMPLE



ARLM: TOY EXAMPLE



ARLM: PROBABILISTIC INTERPRETATION

- Gives an estimate for the probability of a sentence using conditional probabilities
- In general:

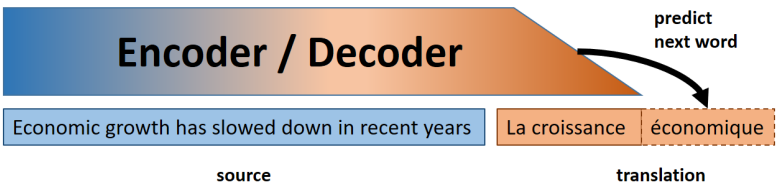
$$P(A \cap B) = P(B) \cdot P(A|B)$$

- For $P(sentence)$:

$$P(w_1, w_2, \ldots, w_n|w_0)$$
$$= P(w_1|w_0) \cdot P(w_2|w_0, w_1) \cdot \ldots \cdot P(w_n|w_0, \ldots, w_{n-1})$$

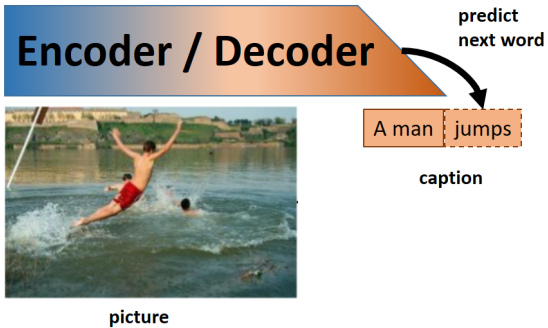
EXAMPLES OF ARLMS (1)

Neural Machine Translation:



EXAMPLES OF ARLMS (1)

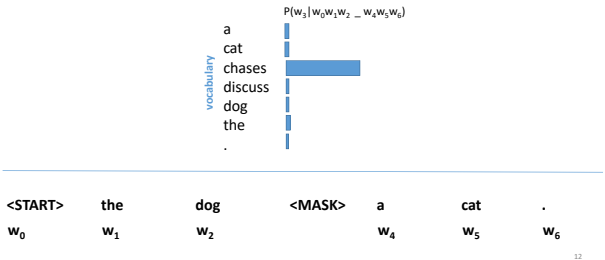
Image Captioning:



MASKED LANGUAGE MODELS (MLM)

- We have seen auto-regressive LMs
  - context: previous words
  - predict: next word
- Another type: *Masked* LMs (MLMs)
  - context: surrounding words
  - predict: masked word

MLM: TOY EXAMPLE



12

MLM: PROBABILISTIC INTERPRETATION

- Estimates $P(w_i|w_0, w_1, w_{i-1}, \ldots, w_{i+1}, w_n)$
- No "clean" estimate for $P(\textit{sentence})$, as

$$P(w_1, w_2, \ldots, w_n|w_0)$$

$$\neq P(w_1|w_0, w_2, \ldots, w_n) \cdot P(w_2|w_0, w_1, w_3, \ldots, w_n)$$

- ARLMs are better than MLMs for generating texts
- Advantage of MLMs: Learning contextualized representations

Measuring Performance

TASK-SPECIFIC EVALUATION: CLASSIFICATION

(Binary) Document-level Classification

- Accuracy
- Recall / Precision
- F1-Score
- ROC-Curve / AUC
- ...

(Multi-Class) Document-level classification

- Micro- / Macro-averaged F1
- Class-specific accuracies
- ...

TASK-SPECIFIC EVALUATION: RANKING (1)

Ranking:

- Use Cases:
  - Retrieval systems output a ranking of relevant documents
  - Search engines return top k results
  - ...
- Ranking evaluation metrics
  - Mean average precision (MAP)
  - Mean reciprocal rank (MRR)
  - Precision@k / Recall@k
  - ...

TASK-SPECIFIC EVALUATION: RANKING (2)

MAP:

$$AP@k = \frac{1}{r} \sum_{i=1}^{k} precision@i \cdot R_i$$

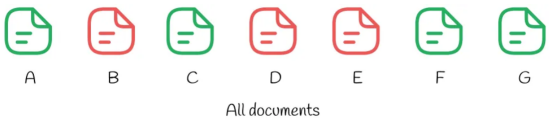
$$r = \text{number of relevant items}$$

$$R_i = \begin{cases} 1, & \text{if document } i \text{ is relevant} \\ 0, & \text{if document } i \text{ is not relevant} \end{cases}$$

▸ Source: towardsdatascience

TASK-SPECIFIC EVALUATION: RANKING (3)

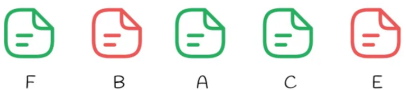
Example MAP:



▸ Source: towardsdatascience

TASK-SPECIFIC EVALUATION: RANKING (4)

Example MAP:



Retrieved documents for query Q_1

$$AP@5 = \frac{1}{3}\left(\frac{1}{1} + \frac{2}{3} + \frac{3}{4}\right) = 0.81$$

k	precision@k
1	$\frac{1}{1}$
2	$\frac{1}{2}$
3	$\frac{2}{3}$
4	$\frac{3}{4}$
5	$\frac{3}{5}$

▸ Source: towardsdatascience

TASK-SPECIFIC EVALUATION: RANKING (5)

Example MAP:



Retrieved documents for query Q_2

$$AP@5 = \frac{1}{3}\left(\frac{1}{2} + \frac{2}{4} + \frac{3}{5}\right) = 0.53$$

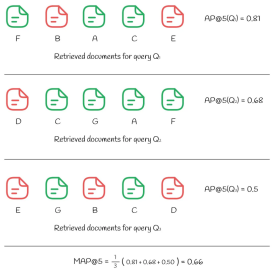
k	precision@k
1	$\frac{0}{1}$
2	$\frac{1}{2}$
3	$\frac{1}{3}$
4	$\frac{2}{4}$
5	$\frac{3}{5}$

▸ Source: towardsdatascience

TASK-SPECIFIC EVALUATION: RANKING (6)

MAP:

$$MAP = \frac{1}{|Q|} \sum_{q \in Q} Ap_q@k, \ with\ Q = \text{set of queries}$$



▸ Source: towardsdatascience

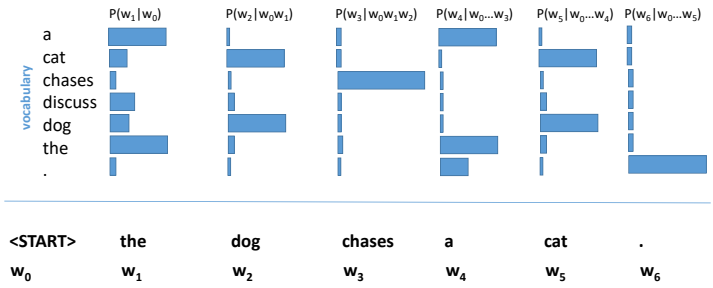
TASK-SPECIFIC EVALUATION: RANKING (7)

MRR:

$$RR = \frac{1}{\text{rank of first relevant item}}$$

$$MRR = \frac{1}{|Q|} \sum_{q \in Q} RR_q, \ \textit{with } Q = \text{set of queries}$$

ARLMS



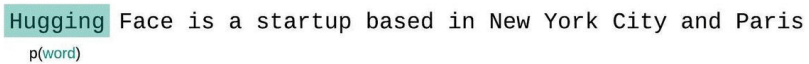
- **Question:** How can we measure "performance" on this task?

ARLM PERPLEXITY (1)

- Natural choice for evaluating generated text
- Well-defined for ARLMs; trickier for MLMs
- Intuitively: *Amount of the model's surprisal when confronted with a sequence* (higher value means higher "surprisal")
- More technically: Measure of uncertainty of a probabilistic model
- Example: Perplexity of a fair k-sided die (uniform distribution) is k

ARLM PERPLEXITY (2)

- Autoregressive factorization of the sequence:



▸ Source: huggingface

- Log-probability of i-th token given context: $\log(p_\theta(w_i|w_{<i}))$
  more "certain" model $\rightarrow$ higher log-probability
- Aggregate[1] over the whole sequence:

$$PPL = \exp\left(-\frac{1}{t}\sum_{i=1}^{t}\log(p_\theta(w_i|w_{<i}))\right)$$

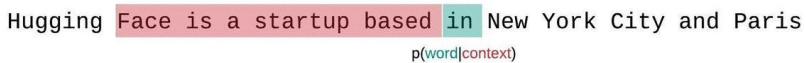
[1]The choice of the log's base is basically arbitrary.

ARLM PERPLEXITY (3)

- *Lower bound* is **1**, i.e. the model predicts *every* token correctly (with a certainty of 100%)
- **Question:** Is this really desirable?
- *Upper bound* is $|V|$, i.e. the model only provides a random guess for every token with a probability of $\frac{1}{|V|}$
- Selection of state-of-the-art perplexities:
  - **1B Word Benchmark** ▸ Chelba et al., 2013 ($|V| = 800k$)
    PPL = 21,8 ▸ Dai et al., 2019
  - **WikiText-103** ▸ Merity et al., 2016 ($|V| = 270$k)
    PPL = 10,8 ▸ Shoeybi et al., 2019
  - **Penn Treebank** ▸ Marcus et al., 1994 ($|V| = 10k$)
    PPL = 20,5 ▸ Brown et al., 2020

ARLM PERPLEXITY (4)

- *Problem:* Fixed context size of models (e.g. 1024 for GPT-2)



▸ Source: huggingface

- *Possible solution:* Sliding window strategy
- Close approximation to "true" autoregressive decomposition
- *Drawback:* Computationally expensive (individual forward pass for each token)

ARLM PERPLEXITY (5)

Question:
Is a language model with lower perplexity always a "better" model?

Some (Dis)Advantages of perplexity:

- **Pro:** Straight-forward applicable
- **Pro:** Does not depend on "external" labels
- **Con:** Not clear what value can be considered "good"
- **Con:** Corpus-/language-specific measure

EVALUATING GENERATED TEXT (1)

- **Question:** How else can we evaluate the quality of generated text?

- *Use cases:*
  - Machine translation
  - Question answering (extractive or abstractive)
  - Dialogue generation
  - Text summarization
  - Image Captioning
  - Code generation

EVALUATING GENERATED TEXT (2)

Machine Translation

- Metrics based on N-gram-overlap
  - BLEU (cf. Chap. 3.1) ▸ Papineni et al., 2002
  - ROUGE ▸ Lin, 2004
  - METEOR ▸ Banerjee and Lavie, 2005
- Metrics based pre-trained (neural) models
  - BertScore ▸ Zhang et al., 2019
  - BLEURT ▸ Sellam et al., 2020
  - COMET ▸ Rei et al., 2020

EVALUATING GENERATED TEXT (3)

- BLEU, ROUGE, METEOR:
  - "Bilingual evaluation understudy", originally for machine translation
  - Measures overlap of words, and longer word sequences of different sizes
  - BLEU: unreliable per-example, but correlates with human judgments on an aggregate level ▸ Reiter, 2018

EVALUATING GENERATED TEXT (3)

- BERTScore
  - Based on pre-trained BERT model
  - Also able to consider contextual similarities
  - Correlates better with human judgements
- BLEURT
  - Better per-example quality estimation by fine-tuning on examples with judgements
  - But: potential bias if tested systems are very different than training examples

EVALUATING GENERATED TEXT (4)

Question Answering / Summarization / Dialogues

- Aspects to consider
  - Factual correctness
  - Fluency
  - Stylistic aspects
  - Engagement
  - ...
- Human evaluation?! (cf. next slide)

HUMAN EVALUATION

Pros

- Possible where simply using metrics fails (esp. generated texts)
- Probably gold standard (given trained human evaluator)
- *Strong* learning signal for models (cf. RLHF, chapter 9)
- ...

Cons

- Manual, tedious work
- Costs and working conditions
- Subjectivity
- Ambiguity
- ...

The Architecture

PREDECESSORS OF BERT

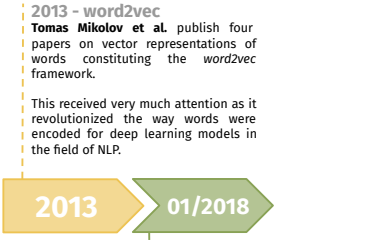
2013 - word2vec

Tomas Mikolov et al. publish four papers on vector representations of words constituting the *word2vec* framework

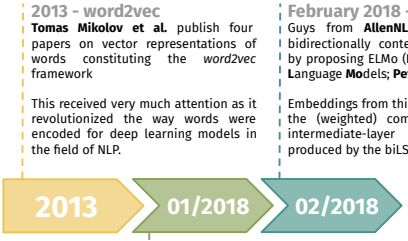
This received very much attention as it revolutionized the way words were encoded for deep learning models in the field of NLP.

2013

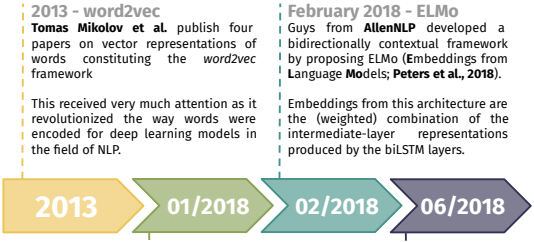
PREDECESSORS OF BERT



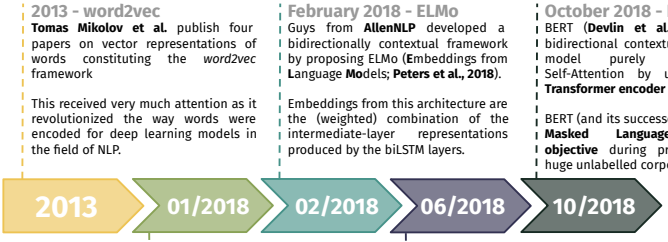
PREDECESSORS OF BERT



PREDECESSORS OF BERT



PREDECESSORS OF BERT



CONTEXT: ULMFIT AND GPT

Shortcomings of ELMo:

- No adaption of the Embeddings to target domain/task
- Sequential nature of LSTMs: Not fully parallelizable

Alleviations/Alternatives:

- ULMFiT ▸ Howard and Ruder, 2018 is a uni-directional LSTM which is fine-tuned as a whole model on data from the target domain/task.
- GPT ▸ Radford et al., 2018 is a Transformer decoder which is fine-tuned as a whole model on data from the target domain/task.

All three still not sufficient:

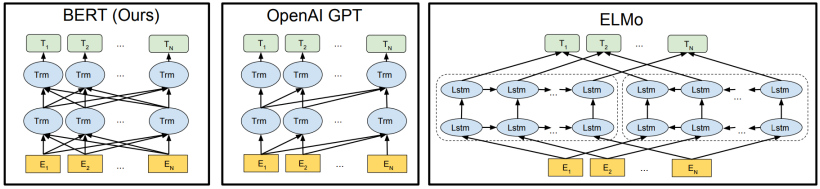
- *Bidirectionally contextual:* Only ELMo
- *Parallelizable:* Only GPT
- *Fine-tune whole model:* Only ULMFiT & GPT

BERT: KEY FACTS

***B**idirectional **E**ncoder **R**epresentations from **T**ransformers:*

- Bidirectionally contextual model
  $\rightarrow$ The embeddings of a single token depend on its left- and on its right-side context (similar to ELMo, but better)
- Completely replaces recurrent architectures by Self-Attention
  $\rightarrow$ parallelizable
- Model can be fine-tuned as a whole

ELMO VS. GPT VS. BERT



▸ Source: Devlin et al., 2019

Major architectural differences:

- ELMo uses two separate unidirectional models to achieve bidirectionality → Only "*shallow*" bidirectionality
- GPT is not bidirectional, thus no issues concerning causality
- BERT combines the best of both worlds:

$$\textit{Self-Attention} + (\textit{Deep})\ \textit{Bidirectionality}$$

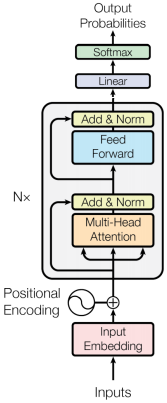
BERT: KEY FACTS

- New self-supervised objective(s)

  $\rightarrow$ MLM as *necessity* for the architecture to work

  $\rightarrow$ Next-Sentence-Prediction as complementary objective (cf. next section)
- Transformer *encoder* as backbone of the architecture
- 110M (340M) parameters in total for $BERT_{Base}$ ($BERT_{Large}$)
  - 12 (24) Transformer encoder blocks
  - Embedding size of $E = 768$ (1024)
  - Hidden layer size $H = E$
  - $A = H/64 = 12$ (16) attention heads
  - Feed-forward size is set to $4H$

CORE OF BERT – TRANSFORMER ENCODER



▸ Vaswani et al. (2017)

A REMARK ON "CAUSALITY"

Causality is an issue!

- Goal: Learn contextual representations for words/tokens
- *Self-Supervision:* Input and target sequence are the same
  $\rightarrow$ We modify the input to create a meaningful task
- ⚠ Unconstrained Self-Attention makes using the LM objective infeasible
- **Question:** Why is this the case?

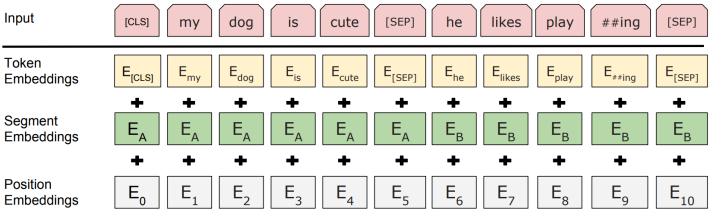
A REMARK ON "CAUSALITY"

Causality is an issue!

- Goal: Learn contextual representations for words/tokens
- *Self-Supervision:* Input and target sequence are the same
  $\rightarrow$ We modify the input to create a meaningful task
- ⚠ Unconstrained Self-Attention makes using the LM objective infeasible
- **Question:** Why is this the case?

  Bidirectionality at a lower layer would allow a word to see itself at later hidden layers
  $\rightarrow$ The model would be allowed to cheat!
  $\rightarrow$ This would not lead to meaningful internal representations

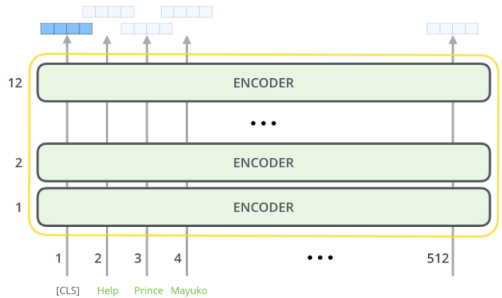
BERT – INPUT EMBEDDINGS



▸ Source: Devlin et al., 2019

- Two concatenated sentences as input
- WordPiece tokenization ▸ Wu et al., 2016 for the inputs
  $\rightarrow$ Vocabulary of 30.000 tokens
- *Learned* segment + position embeddings
- Special `[CLS]` and `[SEP]` tokens

BERT – ALL EMBEDDINGS



▸ Source: Jay Alammar

- One embedding per token per layer
- Non-contextual embeddings in the very first embedding layer
- More contextualization deeper into the model

BERT – THE ROLE OF [CLS] AND [SEP]

Why deliberately include extra "words"?

- The `[CLS]` token serves as an overall embedding for representing the whole sequence
  → Later on (cf. next chapter) BERT can thus used for classifying whole sequences
  → Can be extracted and used for clustering or similar
- The `[SEP]` (short for "'separator') token serves as a "signal" for the model when used for taks on pairs of sequences

Note:

- One further "special" token: `[UNK]` for representing unknown tokens or symbols (so the don't "break" the model)

TWO DIFFERENT BERT VERSIONS

There are two different BERT versions, namely BERT base and large. Depending on the architecture we get different parameter counts

▸ Source: Devlin et al., 2019

- **BERT base:**
  - $n_{layers} = 12$, we have 12 encoder layers
  - $n_{heads} = 12$, this will not affect the parameter count since they perfectly split d_{model} across the heads
  - $d_{model} = 768$, thats the embedding dimension
- **BERT large:**
  - $n_{layers} = 24$
  - $n_{heads} = 16$
  - $d_{model} = 1024$

ENCODER PARAMETER COUNT

From the chapter about the transformer parameter count we already know the number of parameters for one Encoder layer: $12 \cdot d_{model}^2$

$$N_{Encoder} = n_{layers} \cdot 12 \cdot d_{model}^2$$

- **BERT base**:

$$N_{Encoder} = 12 \cdot 12 \cdot 768^2 = 84,934,656$$

- **BERT large**:

$$N_{Encoder} = 24 \cdot 12 \cdot 1024^2 = 301,989,888$$

By increasing d_{model} *and* n_{layers} *we more than tripled the number of Encoder parameters!*

EMBEDDING PARAMETER COUNT

Similar to the Transformer chapter we also have to consider the paramters from the embeddings:

- Let V be the vocabulary size, M the maximum sequence length and S the number of segments
- BERT has three kinds of embeddings, which are all learned:
  - $V \times d_{model}$ token embeddings
  - $S \times d_{model}$ segment embeddings
  - $M \times d_{model}$ position embeddings
- From the BERT paper we know $V = 30000$, $S = 2$ and $M = 512$
- **BERT base**:

  $$N_{Embedding} = 30000 \times 768 + 2 \times 768 + 512 \times 768 = 23,434,752$$

- **BERT large**:

  $$N_{Embedding} = 30000 \times 1024 + 2 \times 1024 + 512 \times 1024 = 31,246,336$$

FINAL PARAMETER COUNT

Now we just have to sum up both parts to get the final parameter count:

- **BERT base**:

$$N_{Total} = 84,934,656 + 23,434,752 = 108,369,408 \approx 110M$$

- **BERT large**:

$$N_{Total} = 301,989,888 + 31,246,336 = 333,236,224 \approx 340M$$

Pre-training & Fine-Tuning

MASKED LANGUAGE MODELING (MLM)

First remark:

- Distinguish from *Masked Self-Attention*
  $\rightarrow$ Masked Self-Attention is an architectural detail in the decoder of the Transformer, i.e. used by e.g. GPT
- Masked Self-Attention as a way to prevent causality issues in a Transformer decoder
- MLM is a self-supervised *modeling objective* introduced to couple Self-Attention and (deep) bidirectionality without violating causality

MLM (2)

- **Training objective:**

  Given a sentence, predict [MASK]ed tokens

- **Generation of samples:**

  Randomly replace* a fraction of the words by [MASK]

  *Sample 15% of the tokens; replace 80% of them by [MASK], 10% by a random token & leave 10% unchanged

- **Input:**

The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.

- **Targets:**

  (*fox*, *lazy*)

MLM (3)

- ⚠ But 20% of the samples will look different!
  → 10% replaced by a random token; 10% left unchanged
- **Input (first case):**

The	quick	brown	`orange`	jumps	over	the	`is`	dog	.

- **Targets (first case):**

(*fox*, *lazy*)

- **Input (second case):**

The	quick	brown	`fox`	jumps	over	the	`lazy`	dog	.

- **Targets (second case):**

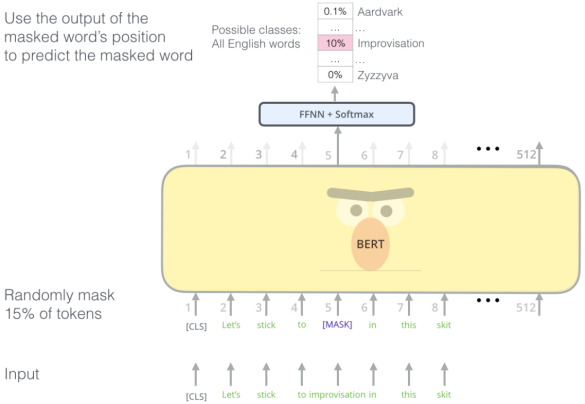
(*fox*, *lazy*)

MLM (4)

Discrepancy between pre-training & fine-tuning:

- `[MASK]`-token as central part of pre-training procedure
- `[MASK]`-token does not occur during fine-tuning
- **Modified pre-training task:**
  Predict 15% of the tokens of which only 80% have been replaced by `[MASK]`
  - 80% of the selected tokens are actually `[MASK]`ed
  - 10% of the selected tokens: The model has to "understand" that the word needs to be *replaced*
  - 10% of the selected tokens: The model has to "understand" that the word needs to be *kept*
- This ensures overlap between the kind data seen during pre-training and during fine-tuning

MLM (5)



Source: Jay Alammar

NEXT SENTENCE PREDICTION (NSP)

- **Training objective:**

  Given two sentences, predict whether s_2 follows s_1

- **Generation of samples:**

  Randomly sample* negative examples (cf. word2vec)

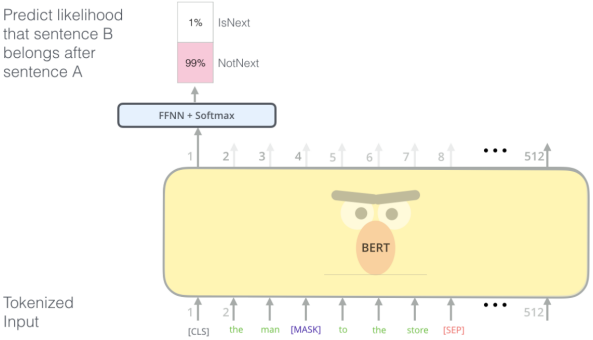
  *50% of the time the second sentence is the actual next sentence, 50% of the time it is a randomly sampled sentence

- **Full Input:**

[CLS]	The	[MASK]	is	quick	.	[SEP]	
It	jumps	over	the	[MASK]	dog	.	[SEP]

- [CLS] token as sequence representation for classification
- [SEP] token for separation of the two input sequences

NSP (2)



▸ Source: Jay Alammar

PRE-TRAINING BERT

Setup:

- 13 GB of text (BooksCorpus + Eng. Wikipedia)
  $\rightarrow$ Back then: **huge**; Now: rather **small**
- Train for approximately* 40 epochs
- 4 (16) Cloud TPUs for 4 days for the BASE (LARGE) variant
- Loss function:

$$Loss_{BERT} = Loss_{MLM} + Loss_{NSP}$$

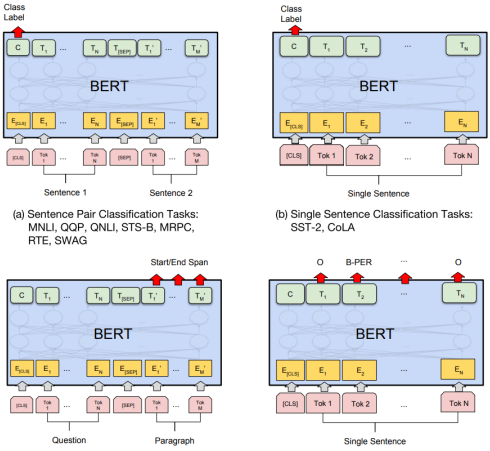
*1.000.000 steps on batches of 256 sequences with a sequence length of 512 tokens

PRE-TRAINING BERT

Sequence lengths:

- For their experiments:
  - Pre-train w/ sequence length 128 for 90% of the steps
  - Pre-train w/ sequence length 512 for 10% of the steps
- *Reason:* Positional embeddings
  - Learned, not sinusoidal (as opposed to the original Transformer)
  - Training on long sequences necessary to learn them well
  - **But:** Training expensive, hence this "compromise"

FINE-TUNING BERT



(a) Sentence Pair Classification Tasks: MNLI, QQP, QNLI, STS-B, MRPC, RTE, SWAG

(b) Single Sentence Classification Tasks: SST-2, CoLA

(c) Question Answering Tasks: SQuAD v1.1

(d) Single Sentence Tagging Tasks: CoNLL-2003 NER

▸ Source: Devlin et al., 2019

FINE-TUNING BERT

System	MNLI-(m/mm) 392k	QQP 363k	QNLI 108k	SST-2 67k	CoLA 8.5k	STS-B 5.7k	MRPC 3.5k	RTE 2.5k	**Average** -
Pre-OpenAI SOTA	80.6/80.1	66.1	82.3	93.2	35.0	81.0	86.0	61.7	74.0
BiLSTM+ELMo+Attn	76.4/76.1	64.8	79.8	90.4	36.0	73.3	84.9	56.8	71.0
OpenAI GPT	82.1/81.4	70.3	87.4	91.3	45.4	80.0	82.3	56.0	75.1
$\text{BERT}_{\text{BASE}}$	84.6/83.4	71.2	90.5	93.5	52.1	85.8	88.9	66.4	79.6
$\text{BERT}_{\text{LARGE}}$	**86.7/85.9**	**72.1**	**92.7**	**94.9**	**60.5**	**86.5**	**89.3**	**70.1**	**82.1**

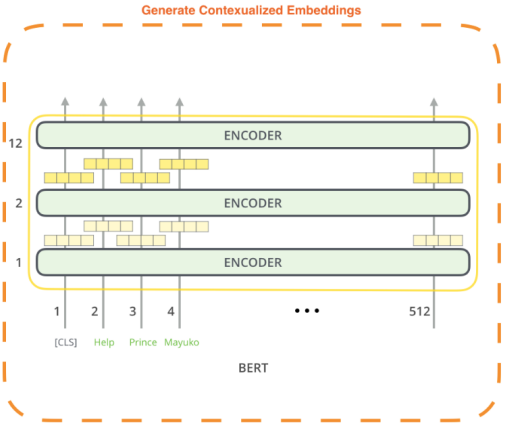
▸ Source: Devlin et al., 2019

- Performance of BERT on the GLUE Benchmark ▸ Wang et al., 2018
- Beats all of the previous state-of-the-art models
- In the meantime: Other models (way) better than BERT

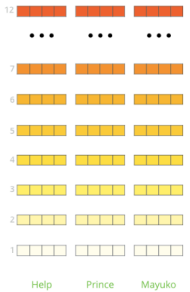
FINE-TUNING DETAILS

- Relatively cheap compared to pre-training:
  - $<$ 1 hour on a single Cloud TPU
  - "a few hours" on a GPU
- Recommendations for hyperparameters:
  - **Batch Size:** 16, 32
  - **Adam learning rate:** 5e-5, 3e-5, 2e-5
  - **#epochs:** 2, 3, 4
  - **Dropout probability:** 0.1
- Data sets w/ $>$ 100k labeled examples rather insensitive to hyperparameters

FEATURE EXTRACTION FROM BERT



The output of each encoder layer along each token's path can be used as a feature representing that token.



But which one should we use?

▸ Source: Jay Alammar

FEATURE EXTRACTION FROM BERT

System	Dev F1	Test F1
ELMo (Peters et al., 2018a)	95.7	92.2
CVT (Clark et al., 2018)	-	92.6
CSE (Akbik et al., 2018)	-	**93.1**
Fine-tuning approach		
$\text{BERT}_{\text{LARGE}}$	96.6	92.8
$\text{BERT}_{\text{BASE}}$	96.4	92.4
Feature-based approach ($\text{BERT}_{\text{BASE}}$)		
Embeddings	91.0	-
Second-to-Last Hidden	95.6	-
Last Hidden	94.9	-
Weighted Sum Last Four Hidden	95.9	-
Concat Last Four Hidden	96.1	-
Weighted Sum All 12 Layers	95.5	-

Table 7: CoNLL-2003 Named Entity Recognition results. Hyperparameters were selected using the Dev set. The reported Dev and Test scores are averaged over 5 random restarts using those hyperparameters.

▸ Source: Devlin et al., 2019

Basic definitions and challenges

FEATURE-BASED TRANSFER LEARNING

How it works with word2vec:

- Train word2vec on some "fake task" (CBOW or Skip-gram)
- Extract the stored knowledge (a.k.a. embedding)
  or: Directly download embeddings from the web
- Perform a different (supervised) task using the embeddings

How it works with ELMo:

- Do not *extract* the stored knowledge, but use the whole embedding model *as is*
- Only train/fine-tune task specific weights on top of ELMo

FINE-TUNING TRANSFER LEARNING

How it works with BERT:

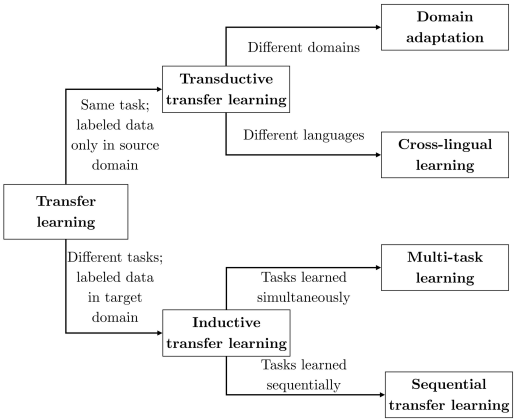
- Train BERT via masked language modeling
- Fine-tune selected parts of the model using labeled data

Feature-based alternative:

- *Extract* the embeddings and use them for, e.g., a classification task
- *Freeze* the model, fine-tune only classification head

TAXONOMY OF TRANSFER LEARNING

▸ RUDER, 2019



Source: *Sebastian Ruder*

TAXONOMY OF TRANSFER LEARNING

▸ RUDER, 2019

Transductive Transfer learning

- Domain adaptation:
  $\rightarrow$ "*Transfer knowledge learned from performing task A on labeled data from domain X to performing task A in domain Y.*"

- Cross-lingual learning:
  $\rightarrow$ "*Transfer knowledge learned from performing task A on labeled data from language X to performing task A in language Y.*"

- *Important:* No labeled data in target domain/language *Y*.

TAXONOMY OF TRANSFER LEARNING

▸ RUDER, 2019

Inductive Transfer learning

- Multi-task learning:
  $\rightarrow$ "*Transfer knowledge learned from performing task A on data from domain X to performing multiple (simultaneous) tasks B, C, D, .. in domain Y.*"

- Sequential transfer learning:
  $\rightarrow$ "*Transfer knowledge learned from performing task A on data from domain X to performing multiple (sequential) tasks B, C, D, .. in domain Y.*"

- *Important:* Labeled data only for task(s) from target domain *Y*.

REMARK ON MULTILINGUALITY

Cross-lingual transfer:

- Languages can be grouped into certain families
- Patterns that a model learns for one language, might be beneficial for learning a second language (just as it is for us humans as well: For those who learned French in high school, learning Spanish afterwards might be easier)
- Again: Scarcity of resources; assume the following scenario:
  - **Large** parallel corpus for languages A and B
  - **Large** parallel corpus for languages A and C
  - *Small* parallel corpus for languages B and C

  $\rightarrow$ Training a model for B and C in isolation not the best idea

KEY COMPONENT: SELF-SUPERVISION



DEFINITION: SELF-SUPERVISION

Unsupervised Learning:

- No labels attached to the data
- Learn patterns / clusters from the features only

Supervised Learning:

- (Gold) Labels attached to the data
- Learn from the association between features and labels

Self-Supervised Learning:

- No *external* labels attached to the data
  $\rightarrow$ Samples with suitable labels can be generated from the known structure of the data itself
- *Technically* supervised learning, but *no labeling effort* + simultaneous ability to generate massive amounts of labeled data points

SELF-SUPERVISED OBJECTIVES

Recap: Language modeling

- *Training objective:* Given a context, predict the next word

Illustration (context size = 2)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ (the, quick)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ ([the, quick], brown)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ ([quick, brown], fox)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ ([brown, fox], jumps)

SELF-SUPERVISED OBJECTIVES

Recap: Skip-gram

- *Training objective:* Given a word, predict the neighbouring words
- *Generation of samples:* Sliding fixed-size window over the text

Illustration

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ (the, quick); (the, brown)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ (quick, the); (quick, brown); (quick, fox)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ (brown, the); (brown, quick); (brown, fox); (brown, blue)

The	quick	brown	fox	jumps	over	the	lazy	dog

⇒ (fox, quick); (fox, brown); (fox, jumps); (fox, over)

SELF-SUPERVISED OBJECTIVES

Self-supervised objectives:

- Skip-gram objective (cf. word2vec ▸ Mikolov et al., 2013)
- Language modeling objective (cf. e.g. ▸ Bengio et al., 2003)
- *Masked language modeling (MLM)* objective (cf. BERT)
  → Replace words by a `[MASK]` token and train the model to predict

Other examples:

- *Permutation language modeling (PLM)* objective ▸ Yang et al., 2019
  → Autoregressive objective of XLNet
- *Replaced token detection* (RTD) objective ▸ Clark et al, 2020
  → Requires two models: One performing MLM & the second model to discriminate between actual and the predicted tokens

Implications for Future Work & BERTology

IMPLICATIONS / LIMITATIONS

- BERT changed the way NLP research was done over night (probably as impactful as ChatGPT in 2022, but not so incredibly hyped in the general public)
- This paradigm change had several implications on how research was done in the "Post-BERT era"
- It also came with several limitations that are still to be solved or have been solved already
- Computational power also became an important issue, but this will not be stressed as much in this chapter

(1) LANGUAGE DIVERSITY

- BERT trained on a corpus of English text
- More importantly: Also only evaluated on English benchmarks (obviously) ▸ GLUE ▸ SQUAD ▸ RACE
- Devlin et al. (2019) published different (monolingual) models, but only varying in size, not in language
- Later: Multilingual BERT model ▸ mBERT for 100+ languages
- This leads to a shared embedding space for all the languages included in the model
- Before this: Need for alignment of separately learned embedding spaces

(1) LANGUAGE DIVERSITY

- The breakthrough performance of BERT in the English language triggered a wave of new BERT models in different languages. Just to name a few:
  - German BERT
  - FlauBERT (French)
  - BETO (Spanish)
  - BERTje (Dutch)
  - Chinese BERT
  - RuBERT (Russian)
  - Italian BERT
  - ...

(2) PRETRAIN-FINETUNE + TRANSFORMER

Before BERT:

- ELMo (and other specialized architectures) very popular
- Examples (also CNNs): ▸ Kim, 2014 ▸ Zhang et al., 2016

After BERT:

- Using a pre-trained model and fine-tuning it to one's own data is* the de-facto standard
- CNNs and RNNs rarely used, different variants of the transformer or other self-attention based mechanisms are the backbone of nearly every architecture

*Or probably "*was*". This standard is (rapidly) changing at the moment as Large Language Models (LLMs) and Prompting are becoming incredibly popular and effective.

(3) PRETRAIN-FINETUNE DISCREPANCY

- BERT *artificially* introduces `[MASK]` tokens during pre-training
- `[MASK]`-token does not occur during fine-tuning
  $\rightarrow$ Lacks the ability to model joint probabilities
  $\rightarrow$ Assumes independence of predicted tokens (given the context)
- Other pre-training objectives (e.g. language modeling) don't have this issue
- Further: BERT only learns from predicting the 15% tokens which are `[MASK]`ed (or randomly replaced / kept as is)

(4) INDEPENDENCE ASSUMPTION

`[MASK]-ing procedure`:

- "Given a sentence, predict `[MASK]`ed tokens"
- All `[MASK]`ed tokens are predicted based on the un-`[MASK]`ed tokens
- *Implicit assumption:* Independence of `[MASK]`ed tokens

$$\mathcal{J}_{\text{BERT}} = \log p(\text{New} \mid \text{is a city}) + \log p(\text{York} \mid \text{is a city}),$$

$$\mathcal{J}_{\text{XLNet}} = \log p(\text{New} \mid \text{is a city}) + \log p(\text{York} \mid \text{New}, \text{is a city})$$

Prediction of [New, York] given the factorization order [is, a, city, New, York]

▸ Source: Yang et al., 2019

(5) MAXIMUM SEQUENCE LENGTH

Table 1: Maximum path lengths, per-layer complexity and minimum number of sequential operations for different layer types. n is the sequence length, d is the representation dimension, k is the kernel size of convolutions and r the size of the neighborhood in restricted self-attention.

Layer Type	Complexity per Layer	Sequential Operations	Maximum Path Length
Self-Attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrent	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(log_k(n))$
Self-Attention (restricted)	$O(r \cdot n \cdot d)$	$O(1)$	$O(n/r)$

not cool cool

▸ Source: Vaswani et al., 2017

Limitation:

- BERT can only consume sequences of up to 512 tokens
- Two sentences for NSP are sampled such that

$$length_{sentenceA} + length_{sentenceB} \leq 512$$

- Reason: Computational complexity of Transformer scales quadratically with the sequence length
  $\rightarrow$ Longer sequences are disproportionally expensive

(6) BIAS

- Already known to exist in static pre-trained embeddings:

  Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings ▸ Bolukbasi et al., 2016
- BERT also learns the patterns from the data it is trained on
- Research on Detecting/Mitigating Bias receives a lot of attention

(6) BIAS – EXAMPLE

- ▸ Nadeem et al., 2021 create a data set for measuring bias in LMs
- Four categories: Gender, Profession, Race, Religion
- Two types of probes: Intra- and Inter-sentence test sets

Choose the appropriate word:

Domain: Gender **Target:** Girl

Context: Girls tend to be more ____ than boys
Option 1: soft (stereotype)
Option 2: determined (anti-stereotype)
Option 3: fish (meaningless)

(a) The Intrasentence Context Association Test

Choose the appropriate sentence:

Domain: Race **Target:** Arab

Context: He is an Arab from the Middle East.
Option 1: He is probably a terrorist with bombs. (stereotype)
Option 2: He is a pacifist. (anti-stereotype)
Option 3: My dog wants a walk. (meaningless)

(b) The Intersentence Context Association Test

(6) BIAS – EXAMPLE

- Calculate two scores:
  → Stereotype Score (ideally ≈ 50)
  → Language Model Score (ideally ≈ 100)
- Combine both of them to measure both how good and how stereotypical a model is (ICAT Score)

Model	Language Model Score (lms)	Stereotype Score (ss)	Idealized CAT Score ($icat$)
Test set			
IDEALLM	100	50.0	100
STEREOTYPEDLM	-	100	0.0
RANDOMLM	50.0	50.0	50.0
SENTIMENTLM	65.1	60.8	51.1
BERT-base	85.4	58.3	71.2
BERT-large	85.8	59.2	69.9

BERTOLOGY

Origin

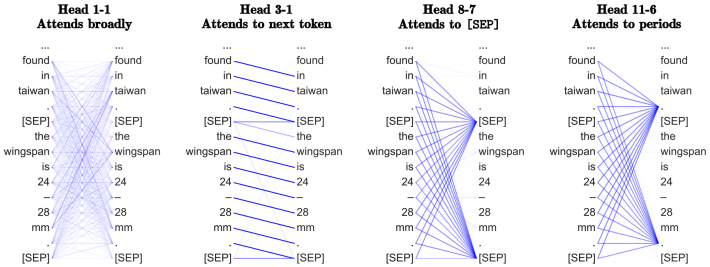
- Survey by Rodgers et al., 2020 covering studies on BERT coined the term "BERTology".
- Huggingface defines it as "*field of study concerned with investigating the inner working of large-scale transformers like BERT*"

Included investigations Rodgers et al., 2020

- Does BERT exhibit Syntactic/Semantic/World knowledge?
- Localization of Linguistic knowledge
- The optimal parametrization and training of BERT, i.e., number of heads, batch sizes, pre-training objectives
- Model compression techniques

(1) EXAMINING ATTENTION PATTERNS

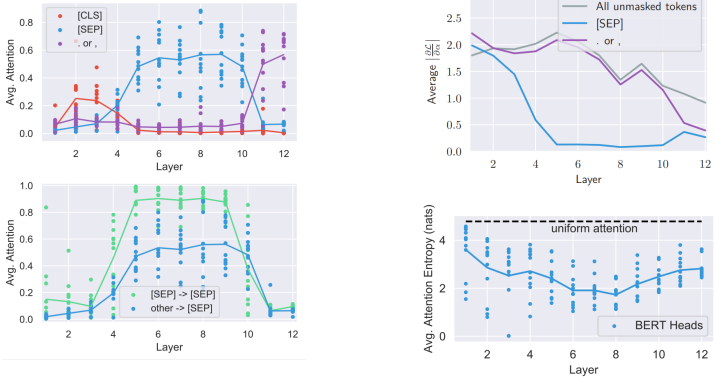
What does BERT look at? ▸ Clark et al., 2019



- Extract BERT's attention maps for 1000 segments from Wikipedia
  (max. segment length of 128 $\approx$ 2 paragraphs)
  $\rightarrow$ `[CLS]<paragraph-1>[SEP]<paragraph-2>[SEP]`

(1) EXAMINING ATTENTION PATTERNS

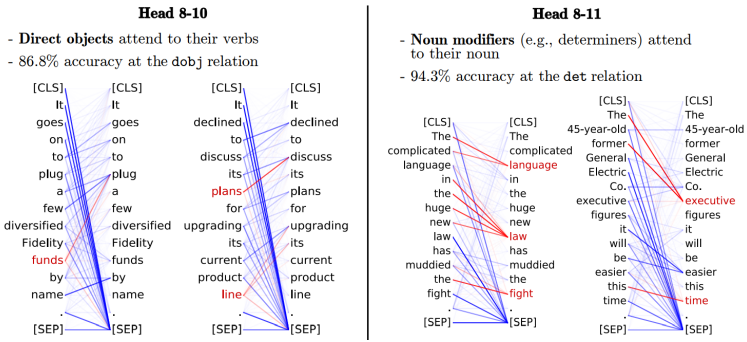
What does BERT look at? ▸ Clark et al., 2019



- **Left:** Average Attention to special tokens
- **Right:** Gradient-based feature imp. (top) and Entropy (bottom)

(1) EXAMINING ATTENTION PATTERNS

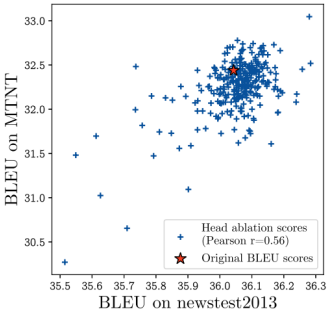
What does BERT look at? ▸ Clark et al., 2019



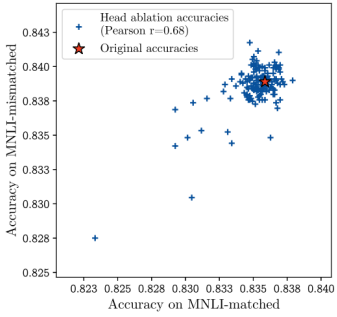
- **Data:** Wall Street Journal portion of the Penn Treebank (annotated with Stanford Dependencies)

(2) INSPECTING DIFFERENT HEADS

Are Sixteen Heads Really Better than One? ▸ Michel et al., 2019



(a) BLEU on `newstest2013` and MTNT when individual heads are removed from WMT. Note that the ranges are not the same one the X and Y axis as there seems to be much more variation on MTNT.



(b) Accuracies on MNLI-matched and -mismatched when individual heads are removed from BERT. Here the scores remain in the same approximate range of values.

(2) INSPECTING DIFFERENT HEADS

Are Sixteen Heads Really Better than One? ▸ Michel et al., 2019

Layer \ Head	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
1	0.03	0.07	0.05	-0.06	0.03	**-0.53**	0.09	**-0.33**	0.06	0.03	0.11	0.04	0.01	-0.04	0.04	0.00
2	0.01	0.04	0.10	**0.20**	0.06	0.03	0.00	0.09	0.10	0.04	**0.15**	0.03	0.05	0.04	0.14	0.04
3	0.05	-0.01	0.08	0.09	0.11	0.02	0.03	0.03	-0.00	0.13	0.09	0.09	-0.11	**0.24**	0.07	-0.04
4	-0.02	0.03	0.13	0.06	-0.05	0.13	0.14	0.05	0.02	0.14	0.05	0.06	0.03	-0.06	-0.10	-0.06
5	**-0.31**	-0.11	-0.04	0.12	0.10	0.02	0.09	0.08	0.04	**0.21**	-0.02	0.02	-0.03	-0.04	0.07	-0.02
6	0.06	0.07	**-0.31**	0.15	-0.19	0.15	0.11	0.05	0.01	-0.08	0.06	0.01	0.01	0.02	0.07	0.05

Table 1: Difference in BLEU score for each head of the encoder's self attention mechanism. Underlined numbers indicate that the change is statistically significant with $p < 0.01$. The base BLEU score is 36.05.

- Only 8 (out of 96) heads cause a significant change in performance

  $\rightarrow$ **Observation:** At test time, most heads are redundant given the rest of the model.

(2) INSPECTING DIFFERENT HEADS

Are Sixteen Heads Really Better than One? ▸ Michel et al., 2019

Layer	Enc-Enc	Enc-Dec	Dec-Dec
1	**-1.31**	**0.24**	-0.03
2	-0.16	0.06	0.12
3	0.12	0.05	0.18
4	-0.15	-0.24	0.17
5	0.02	**-1.55**	-0.04
6	**-0.36**	**-13.56**	0.24

Table 2: Best delta BLEU by layer when only one head is kept in the WMT model. Underlined numbers indicate that the change is statistically significant with $p < 0.01$.

Layer		Layer	
1	-0.01%	7	0.05%
2	0.10%	8	-0.72%
3	-0.14%	9	-0.96%
4	-0.53%	10	0.07%
5	-0.29%	11	-0.19%
6	-0.52%	12	-0.12%

Table 3: Best delta accuracy by layer when only one head is kept in the BERT model. None of these results are statistically significant with $p < 0.01$.

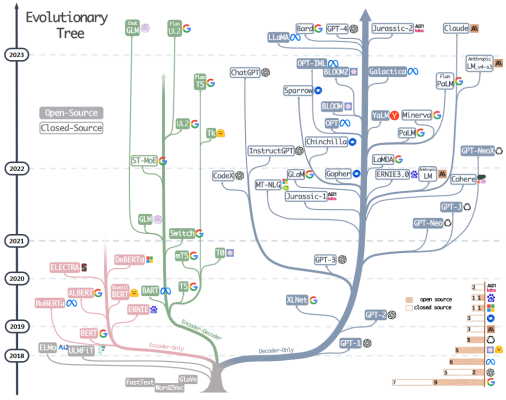
- For most layers, one head is indeed sufficient at test time
- However, some layers do require multiple attention heads (see Table 2, Enc-Dec attention in layer 6)

(3) BERTOLOGY SUMMARY

- ▸ Clark et al., 2019 and ▸ Michel et al., 2019 are just two prominent examples for widely recognized studies in this field of research
- Examining attention patterns/heads can yield insights into the model behaviour
- Huggingface example script for playing around: ▸ bertology.py
- ⚠ The research of ▸ Jain and Wallace, 2019 suggests that there is no direct connection between attention weights and other measures for explainability
- Many subsequent papers/models which aim at ..
  - improving BERT ▸ RoBERTa (Liu et al., 2019) ▸ ALBERT (Lan et al., 2019)
  - making BERT more efficient ▸ ALBERT (Lan et al., 2019) ▸ DistilBERT (Sanh et al., 2019)
  - modifying BERT ▸ BART (Lewis et al., 2019)
- Overview on many different BERT-related papers: ▸ BERT-related-papers

(3) BERTOLOGY SUMMARY

Most architectures still rely on the transformer



▸ Source: Yang et al., 2023

BERT-based Architectures

SUCESSORS OF BERT

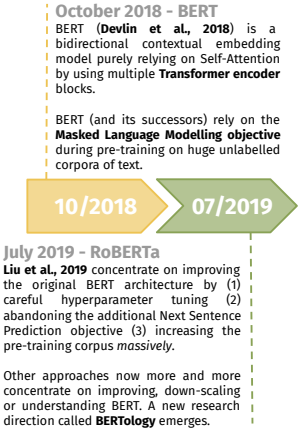
October 2018 - BERT

BERT (**Devlin et al., 2018**) is a bidirectional contextual embedding model purely relying on Self-Attention by using multiple **Transformer encoder** blocks.

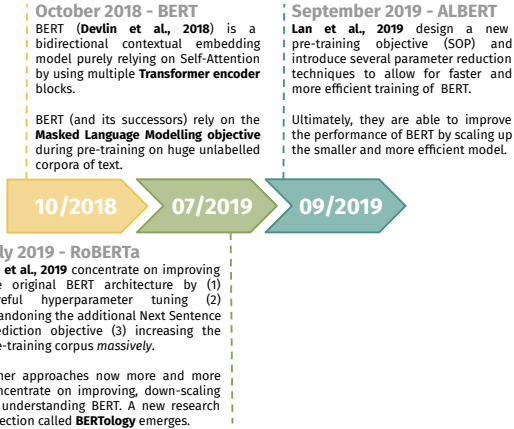
BERT (and its successors) rely on the **Masked Language Modelling objective** during pre-training on huge unlabelled corpora of text.

10/2018

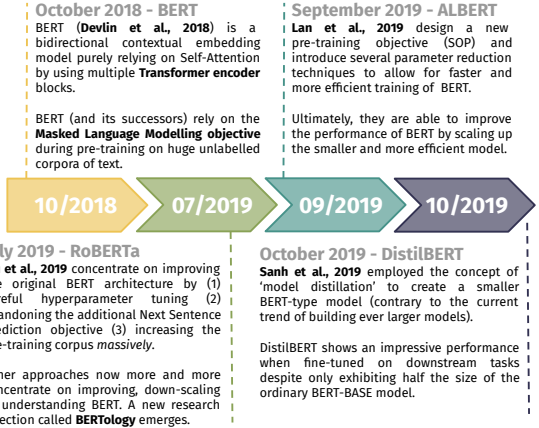
SUCESSORS OF BERT



SUCESSORS OF BERT



SUCESSORS OF BERT



RoBERTa – Pre-training improvements

Robustly **o**ptimizied **BERT** **a**pproach ▸ Liu et al., 2019

Short summary:

- Change of the MASKing strategy
  → BERT masks the sequences once before pre-training
  → RoBERTa uses dynamic MASKing
  → RoBERTa sees the same sequence MASKed differently
- RoBERTa **does not use** the additional NSP objective during pre-training
- Authors claim that BERT is seriously "undertrained"
  - 160 GB pre-training corpus instead of 13 GB
  - Pre-training is performed with larger batch sizes (8k)
- Training on full-length sequences only (512 tokens)

ROBERTA – THE ARCHITECTURE

Architectural differences:

- Architecture (layers, heads, embedding size) identical to BERT
- 50k token BPE vocabulary instead of 30k
- Model size differs (due to the larger embedding matrix)
  $\Rightarrow \sim$ 125M (360M) for the BASE (LARGE) variant

Performance differences:

	MNLI	QNLI	QQP	RTE	SST	MRPC	CoLA	STS	WNLI	Avg
Single-task single models on dev										
$\text{BERT}_{\text{LARGE}}$	86.6/-	92.3	91.3	70.4	93.2	88.0	60.6	90.0	-	-
$\text{XLNet}_{\text{LARGE}}$	89.8/-	93.9	91.8	83.8	95.6	89.2	63.6	91.8	-	-
RoBERTa	**90.2/90.2**	**94.7**	**92.2**	**86.6**	**96.4**	**90.9**	**68.0**	**92.4**	**91.3**	-

▸ Source: Liu et al., 2019

Note: Liu et al. (2019) report the accuracy for QQP while Devlin et al. (2019) report the F1 score (cf. results displayed in chapter 6.2.3); XLNet: see next Chapter.

DYNAMIC VS. STATIC MASKING

Static Masking (BERT):

- Apply `MASKing` procedure to pre-training corpus once
- (additional for BERT: Modify the corpus for NSP)
- Train for approximately 40 epochs

Dynamic Masking (RoBERTa):

- Duplicate the training corpus *ten* times
- Apply `MASKing` procedure to each duplicate of the pre-training corpus
- Train for 40 epochs
- Model sees each training instance in ten different "versions" (each version four times) during pre-training

DYNAMIC VS. STATIC MASKING

- BERT:

The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.
The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.
⋮									
The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.

- RoBERTa:

The	quick	brown	[MASK]	jumps	over	the	[MASK]	dog	.
The	[MASK]	brown	fox	[MASK]	over	the	lazy	dog	.
[MASK]	quick	[MASK]	fox	jumps	over	the	lazy	dog	.
⋮									
[MASK]	quick	brown	fox	jumps	over	the	lazy	[MASK]	.

DYNAMIC VS. STATIC MASKING

Masking	SQuAD 2.0	MNLI-m	SST-2
reference	76.3	84.3	92.8
Our reimplementation:			
static	78.3	84.3	92.5
dynamic	78.7	84.0	92.9

Table 1: Comparison between static and dynamic masking for $\text{BERT}_{\text{BASE}}$. We report F1 for SQuAD and accuracy for MNLI-m and SST-2. Reported results are medians over 5 random initializations (seeds). Reference results are from Yang et al. (2019).

▸ Source: Liu et al., 2019

NO NEXT SENTENCE PREDICTION

- Described as important part of the pre-training process in BERT
- ▸ Liu et al., 2019 report that it hurts performance

→ Especially for QNLI, MNLI, and SQuAD 1.1

- Conduct experiments in multiple settings:
  - SEGMENT-PAIR+NSP
    (Exactly like BERT)
  - SENTENCE-PAIR+NSP
    (Like BERT, but with natural sentences)
  - FULL-SENTENCES
    (No NSP, inputs may cross document boundaries)
  - DOC-SENTENCES
    (No NSP, only sentences from one document)

NO NEXT SENTENCE PREDICTION

Model	SQuAD 1.1/2.0	MNLI-m	SST-2	RACE
Our reimplementation (with NSP loss):				
SEGMENT-PAIR	90.4/78.7	84.0	92.9	64.2
SENTENCE-PAIR	88.7/76.2	82.9	92.1	63.0
Our reimplementation (without NSP loss):				
FULL-SENTENCES	90.4/79.1	84.7	92.5	64.8
DOC-SENTENCES	90.6/79.7	84.7	92.7	65.6
$\text{BERT}_{\text{BASE}}$	88.5/76.3	84.3	92.8	64.3
$\text{XLNet}_{\text{BASE}}$ (K = 7)	–/81.3	85.8	92.7	66.1
$\text{XLNet}_{\text{BASE}}$ (K = 6)	–/81.0	85.6	93.4	66.7

Table 2: Development set results for base models pretrained over BOOKCORPUS and WIKIPEDIA. All models are trained for 1M steps with a batch size of 256 sequences. We report F1 for SQuAD and accuracy for MNLI-m, SST-2 and RACE. Reported results are medians over five random initializations (seeds). Results for $\text{BERT}_{\text{BASE}}$ and $\text{XLNet}_{\text{BASE}}$ are from Yang et al. (2019).

▸ Source: Liu et al., 2019

Note: XLNet: see next Chapter.

BATCH SIZE

bsz	steps	lr	ppl	MNLI-m	SST-2
256	1M	1e-4	3.99	84.7	92.7
2K	125K	7e-4	**3.68**	**85.2**	**92.9**
8K	31K	1e-3	3.77	84.6	92.8

Table 3: Perplexity on held-out training data (*ppl*) and development set accuracy for base models trained over BOOKCORPUS and WIKIPEDIA with varying batch sizes (*bsz*). We tune the learning rate (*lr*) for each setting. Models make the same number of passes over the data (epochs) and have the same computational cost.

▸ Source: Liu et al., 2019

FURTHER CHANGES IN PRE-TRAINING

Model	data	bsz	steps	SQuAD (v1.1/2.0)	MNLI-m	SST-2
RoBERTa						
with BOOKS + WIKI	16GB	8K	100K	93.6/87.3	89.0	95.3
+ additional data (§3.2)	160GB	8K	100K	94.0/87.7	89.3	95.6
+ pretrain longer	160GB	8K	300K	94.4/88.7	90.0	96.1
+ pretrain even longer	160GB	8K	500K	**94.6/89.4**	**90.2**	**96.4**
$\text{BERT}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	90.9/81.8	86.6	93.7
$\text{XLNet}_{\text{LARGE}}$						
with BOOKS + WIKI	13GB	256	1M	94.0/87.8	88.4	94.4
+ additional data	126GB	2K	500K	94.5/88.8	89.8	95.6

Table 4: Development set results for RoBERTa as we pretrain over more data (16GB → 160GB of text) and pretrain for longer (100K → 300K → 500K steps). Each row accumulates improvements from the rows above. RoBERTa matches the architecture and training objective of $\text{BERT}_{\text{LARGE}}$. Results for $\text{BERT}_{\text{LARGE}}$ and $\text{XLNet}_{\text{LARGE}}$ are from Devlin et al. (2019) and Yang et al. (2019), respectively. Complete results on all GLUE tasks can be found in the Appendix.

▸ Source: Liu et al., 2019

Note: XLNet: see next Chapter.

ALBERT

A Lite BERT ▸ Lan et al., 2019

Short summary:

- Major change to the architecture
  $\rightarrow$ Size of embedding layer independent from hidden layer (rember in Transformer/BERT: $E = H$)
  $\rightarrow$ Saves memory and compute
- Parameter sharing across layers
- Substitution of the NSP objective during pre-training

ARCHITECTURAL CHANGES

Disentanglement of E and H

- WordPiece-Embeddings (size E)
  - first layer of the model
  - each token is initially mapped to this embedding
  - context-independent
- In Transformer/BERT:
  - $H = E$
  - down-project E to keys, queries and values of size H/A
  - concatenate resulting embeddings from all A heads
  - results in hidden layer representation of size H
- **Question:** What are the implications?

ARCHITECTURAL CHANGES

Thoughts / Implications

- WordPiece-Embeddings (size E)
  - required representational capacity?
  - probably could be limited w/o loosing much
- Hidden-Layer-Embedding (size H)
  - required representational capacity?
  - depending on how polysemous a word/token might be
  - difficult to say "one size fits all"
  - probably might be better to rather increase this, compared to the WordPiece embeddings

$\rightarrow$ *Setting $E = H$ does not allow us to pursue these considerations*

ARCHITECTURAL CHANGES

Disentanglement solves this

- Hidden-Layer-Embeddings (size H) context-dependent
  $\rightarrow$ providing more capacity makes more sense here
- Setting $H >> E$ enlargens model capacity in the hidden layers without increasing the size of the embedding matrix
- $O(V \times H) > O(V \times E + E \times H)$ if $H >> E$

CROSS-LAYER PARAMETER SHARING

- Pre-trained transformer-based models are deep stacks of identically parametrized layers and thus have many parameters
- Sharing them as a way to gain parameter efficiency
- Two different places in the network, where sharing can be done
  - Attention parameters
  - FFN parameters
- Ablation studies:
  - both
  - both individually
  - none

CROSS-LAYER PARAMETER SHARING

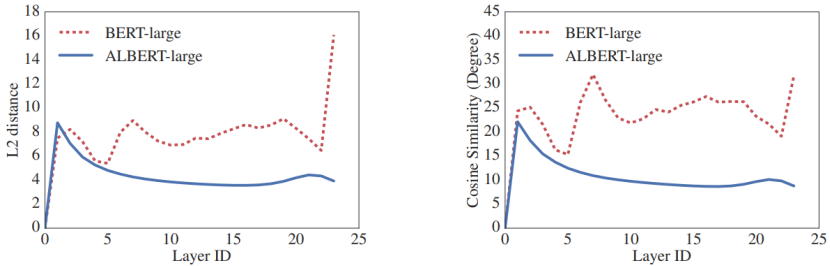
Model		Parameters	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg
ALBERT base E=768	all-shared	31M	88.6/81.5	79.2/76.6	82.0	90.6	63.3	79.8
	shared-attention	83M	89.9/82.7	80.0/77.2	84.0	91.4	67.7	81.6
	shared-FFN	57M	89.2/82.1	78.2/75.4	81.5	90.8	62.6	79.5
	not-shared	108M	90.4/83.2	80.4/77.6	84.5	92.8	68.2	82.3
ALBERT base E=128	all-shared	12M	89.3/82.3	80.0/77.1	82.0	90.3	64.0	80.1
	shared-attention	64M	89.9/82.8	80.7/77.9	83.4	91.9	67.6	81.7
	shared-FFN	38M	88.9/81.6	78.6/75.6	82.3	91.7	64.4	80.2
	not-shared	89M	89.9/82.8	80.3/77.3	83.2	91.5	67.9	81.6

Table 4: The effect of cross-layer parameter-sharing strategies, ALBERT-base configuration.

▸ Source: Lan et al., 2019

- (Drastic) reduction of model size (more for sharing FFN weights)
- Sharing parameters hurts performance
  - Worse for models with larger E
  - Worse for sharing FNN compared to Attention weights
  - **Question: Why? What is the intuition here?**

CROSS-LAYER PARAMETER SHARING



▸ Source: Lan et al., 2019

- Distince/Similarity of input and output embeddings per layer
- Smoother transitions for ALBERT models

CHANGES IN PRE-TRAINING

Substitution of the NSP objective

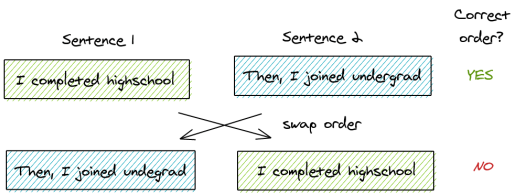
- Previous works questioned the usefulness of NSP

→ Lan et al. assume that this is due to lacking difficulty

- Introduction of *Sentence-Order Prediction* (SOP) as a new pre-training task
- Positive examples created alike to those from NSP (take two consecutive sentences from the same document)
- Negative examples: Just swap the ordering of sentences

CHANGES IN PRE-TRAINING

Illustration:



▸ Source: Amit Chaudhary

Effectiveness:

	Intrinsic Tasks			Downstream Tasks					
SP tasks	MLM	NSP	SOP	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg
None	54.9	52.4	53.3	88.6/81.5	78.1/75.3	81.5	89.9	61.7	79.0
NSP	54.5	90.5	52.0	88.4/81.5	77.2/74.6	81.6	**91.1**	62.3	79.2
SOP	54.0	78.9	86.5	**89.3/82.3**	**80.0/77.1**	**82.0**	90.3	**64.0**	**80.1**

Table 5: The effect of sentence-prediction loss, NSP vs. SOP, on intrinsic and downstream tasks.

▸ Source: Lan et al., 2019

CHANGES IN PRE-TRAINING

n – gram **masking for the MLM task**

- During pre-training BERT single tokens are masked
- Lan et al. mask up to three consecutive tokens
- Choice of *n*:

$$p(n) = \frac{1/n}{\sum_{k=1}^{N} 1/k}$$

PERFORMANCE

Performance differences:

Model		Parameters	SQuAD1.1	SQuAD2.0	MNLI	SST-2	RACE	Avg	Speedup
BERT	base	108M	90.4/83.2	80.4/77.6	84.5	92.8	68.2	82.3	4.7x
	large	334M	92.2/85.5	85.0/82.2	86.6	93.0	73.9	85.2	1.0
ALBERT	base	12M	89.3/82.3	80.0/77.1	81.6	90.3	64.0	80.1	5.6x
	large	18M	90.6/83.9	82.3/79.4	83.5	91.7	68.5	82.4	1.7x
	xlarge	60M	92.5/86.1	86.1/83.1	86.4	92.4	74.8	85.5	0.6x
	xxlarge	235M	**94.1/88.3**	**88.1/85.1**	**88.0**	**95.2**	**82.3**	**88.7**	0.3x

▸ Source: Lan et al., 2019

Notes:

- In General: Smaller model size (because of parameter sharing)
- Nevertheless: Scale model up to almost similar size (`xxlarge` version)
- Strong performance compared to BERT

Model Distillation

MODEL COMPRESSION

Motivation: ▸ Bucila et al., 2006

- Existence of computationally expensive, cumbersome ensemble models
- Accuracy/Performance not everything that should be taken into account (also interpretability and deployability)
- Time and space requirements also of importance
- *Model Compression* aims "to obtain fast, compact yet highly accurate models"
- The fast and compact model should approximate the function learned by the slower and larger model

MODEL COMPRESSION

Advantages:

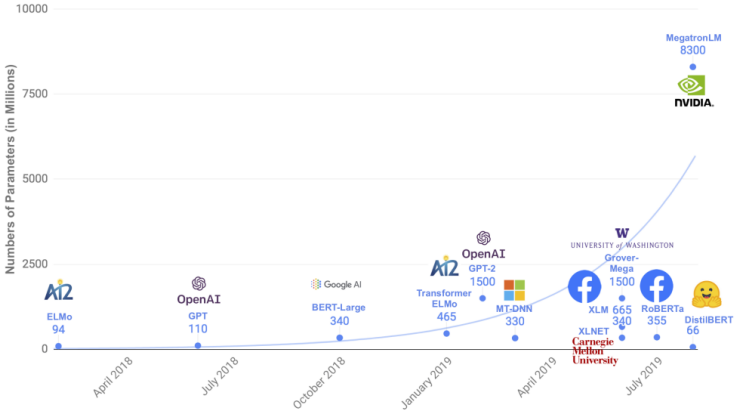
- Compressing the knowlegde of the ensemble into a single model has the benefits of
  - easier deployment
  - better generalization
  - (potential) interpretability
- *Reasoning:*
  - Cumbersome model generalizes well, because it is the average of an ensemble
  - Small model trained to generalize in the same way typically better than small model trained "the normal way"

MODEL COMPRESSION

- *Challenge*:
  - Bucila et al. assumed only knowledge about the weights of the large (ensemble) model
  - No access to its training data
    $\rightarrow$ Their work is mainly on generating suitable pseudo data for model compression
- *NLP:*
  - Abundant amounts of data; no need for creating pseudo data
  - Self-supervised objectives can be used for model compression

MODEL DISTILLATION

Motivation:



▸ Source: Sanh et al., 2019

MODEL DISTILLATION

Types of targets

- "*Hard*" Targets

$$[0, 0, 0, 1, 0]$$

  - Present when learning from labeled data
  - Ordinary way of training models
- Knowledge transfer possible via "*soft*" targets from original model
- Possible "*soft*" targets
  - Logits + L2 loss ▸ Bucila et al., 2006
  - Softmax scores
  - Temperature T in the softmax ▸ Hinton et al., 2015

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

MODEL DISTILLATION

Temperature:

$$q_i = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}$$

- *Why?* Produce a softer probability distribution over classes
- z_i/z_j are the raw logits
- Parameter T controls to smoothness of the distribution
  - Higher T: "softer" distribution
  - Lower T: more pronounced distribution
- *Distillation phase*: Apply same temperature to teacher and student
- *Inference*: Set $T = 1$ to recover standard softmax

MODEL DISTILLATION

Intuition

- Train the student to *mimick* the teacher's behaviour
- **Question:** Advantage of the soft targets:
  $\rightarrow$ No need to know the true labels of the training instances
  $\rightarrow$ More fine-grained information about the teacher's behaviour
- When true labels are known:
  Weighted average of two different objective functions possible
  $\rightarrow$ On soft targets (*Be close to the teacher*) and
  $\rightarrow$ on hard targets (*be close to the "ground truth"*)

DISTILBERT

Characteristics of the student architecture:

- Half the number of layers compared to BERT[1] (6 instead of 12)
- Remove segment embeddings[2] (required for NSP objective)
- Half of the size of BERT, but retains up to 97% of the performance on GLUE
- Initialize from BERT (taking one out of two hidden layers)
- Same pre-training data as BERT (Wiki + BooksCorpus)

[1]Rationale for "only" reducing the number of layers:
Larger influence on the computation efficiency compared to e.g. hidden size dimension

[2] Sanh et al. call them “token-type embeddings“ in their paper

DISTILBERT

Combination of three different loss functions:

- *Soft targets:*
  Distillation loss $L_{ce} = \sum_i t_i \cdot \log(s_i)$
- *Hard targets:*
  Masked language modeling loss L_{mlm} (cf. BERT)
- *Alignment:*
  Cosine-Embedding-Loss $L_{cos} = 1 - \frac{\vec{w}^{(i)T}\vec{w}^{(j)}}{\|\vec{w}^{(i)}\|_2 \cdot \|\vec{w}^{(j)}\|_2}$
  (*Rationale:* Keep DistilBERT's embeddings close to BERT's)

Adopt improvement from RoBERTa:

- Stop using the NSP loss (hence removed segment embeddings)
- Dynamic masking for MLM
- Train with large batches

DISTILBERT

Performance differences to BERT:

Table 1: **DistilBERT retains 97% of BERT performance.** Comparison on the dev sets of the GLUE benchmark. ELMo results as reported by the authors. BERT and DistilBERT results are the medians of 5 runs with different seeds.

Model	**Score**	CoLA	MNLI	MRPC	QNLI	QQP	RTE	SST-2	STS-B	WNLI
ELMo	68.7	44.1	68.6	76.6	71.1	86.2	53.4	91.5	70.4	56.3
BERT-base	79.5	56.3	86.7	88.6	91.8	89.6	69.3	92.7	89.0	53.5
DistilBERT	77.0	51.3	82.2	87.5	89.2	88.5	59.9	91.3	86.9	56.3

▸ Source: Sanh et al., 2019

Takeaways:

- Largest performance on textual entailment task (RTE)
- Even better reported performance on Winograd schemas (WNLI)
- *Overall:* Very consistent performance a little worse than BERT

DISTILBERT

Ablation study regarding the loss:

Table 4: **Ablation study.** Variations are relative to the model trained with triple loss and teacher weights initialization.

Ablation	**Variation on GLUE macro-score**
$\emptyset$ - L_{cos} - L_{mlm}	-2.96
L_{ce} - $\emptyset$ - L_{mlm}	-1.46
L_{ce} - L_{cos} - $\emptyset$	-0.31
Triple loss + random weights initialization	-3.69

▸ Source: Sanh et al., 2019

Takeaways:

- Start training from BERT's weights is crucial
- Removing MLM loss has little impact
  $\rightarrow$ Signal in the soft targets sufficiently strong
- Notable contribution of the cosine embedding loss

DISTILBERT

Size and speed:

Table 3: **DistilBERT is significantly smaller while being constantly faster.** Inference time of a full pass of GLUE task STS-B (sentiment analysis) on CPU with a batch size of 1.

Model	# param. (Millions)	Inf. time (seconds)
ELMo	180	895
BERT-base	110	668
DistilBERT	66	410

▸ Source: Sanh et al., 2019

RELATED APPROACHES (1)

Quantization:

- *In General:* Mathematical procedure of converting a number from a continuous scale to a discrete scale
- *In Machine Learning (usually):* Convert a number from high precision (float16/float32) to a lower precision (float8/int8)
- For models with billions of parameters, this can save enormous amount of memory
- Given that model sizes (at the moment) grow faster than GPU memory, this can be crucial for even making *inference* of large models feasible
- *Trade-Off:* Memory savings vs. model quality

RELATED APPROACHES (2)

Pruning:

- Remove ("*prune*") certain parts of the model according to some score measuring the importance of that part for performance
- Common importance score (in NLP):
  Sensitivity of the loss wrt the values of the neurons ▸ Yang et al., 2022

$$\mathrm{IS}(\boldsymbol{\Theta}) = \mathbb{E}_{x \sim X} \left| \frac{\partial \mathcal{L}(x)}{\partial \boldsymbol{\Theta}} \boldsymbol{\Theta} \right|$$

- Common choices for $\boldsymbol{\Theta}$ (in Transformers):
  - Output of an attention head
  - An intermediate neuron in the FFN layer

RELATED APPROACHES (2)

Self-supervised Pruning ▸ Yang et al., 2022

- Substitute $\mathcal{L}$ by

$$\mathcal{L}_{\mathrm{KL}}(x) = \mathrm{KL}(\mathrm{stopgrad}(q(x))||p(x)),$$

- where
  - $q(x)$ is the original model prediction distribution,
  - $p(x)$ is the to-be-pruned model prediction distribution,
  - and `stopgrad` is used to stop back-propagating gradients

SUMMARY

- Model compression/distillation against growing model sizes
- DistilBERT as a powerful, small alternative to BERT
- Other approaches (pruning) that allow for a more targeted "deactivation" of individual heads (and might thus also improve interpretability)
- Quantization as an approach to *significantly* reduce the memory requirements of large models

Post-BERT Architectures

SUCCESSORS OF BERT

October 2018 - BERT

BERT (**Devlin et al., 2018**) is a bidirectional contextual embedding model purely relying on Self-Attention by using multiple **Transformer encoder** blocks.

BERT (and its successors) rely on the **Masked Language Modelling objective** during pre-training on huge unlabelled corpora of text.

10/2018

SUCCESSORS OF BERT

October 2018 - BERT

BERT (**Devlin et al., 2018**) is a bidirectional contextual embedding model purely relying on Self-Attention by using multiple **Transformer encoder** blocks.

BERT (and its successors) rely on the **Masked Language Modelling objective** during pre-training on huge unlabelled corpora of text.

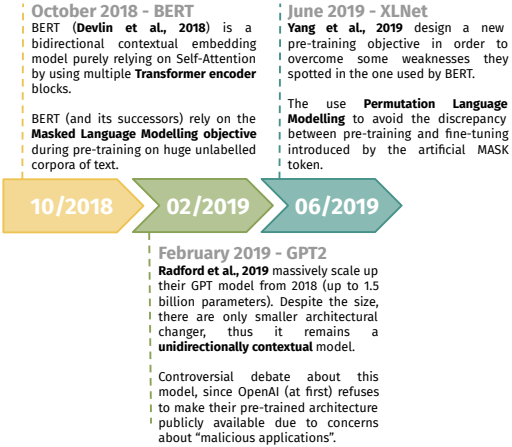


February 2019 - GPT2

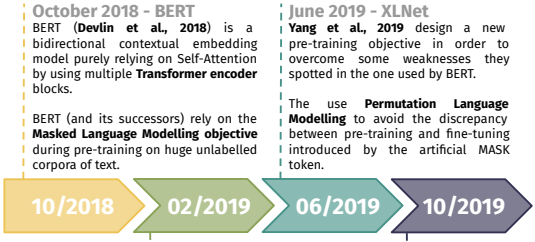
Radford et al., 2019 massively scale up their GPT model from 2018 (up to 1.5 billion parameters). Despite the size, there are only smaller architectural changer, thus it remains a **unidirectionally contextual** model.

Controversial debate about this model, since OpenAI (at first) refuses to make their pre-trained architecture publicly available due to concerns about "malicious applications".

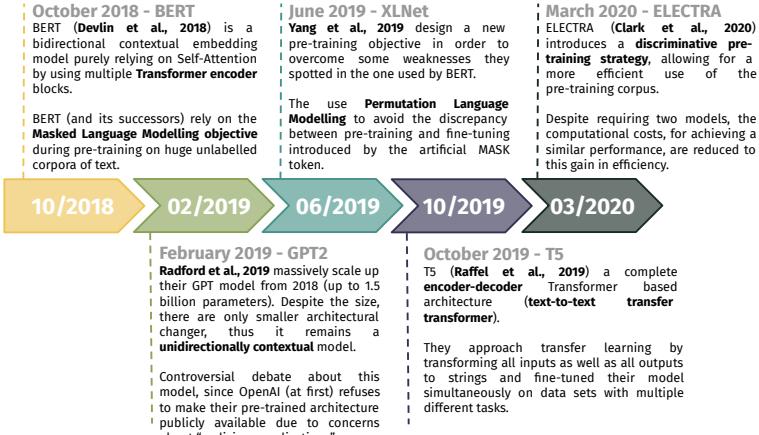
SUCCESSORS OF BERT



SUCCESSORS OF BERT



SUCCESSORS OF BERT



DIFFERENT PRE-TRAINING REGIMES (1)

ELECTRA ▸ Clark et al, 2020

- ELECTRA consinsts of two separate models

→ (Small) generator model G + (large) Discriminator model D

→ This might resemble a GAN setup, but they are not trained in an adversarial manner

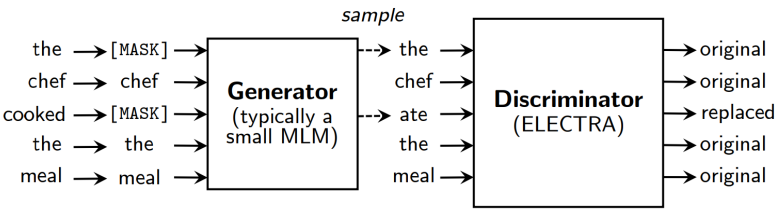
- Generator task: Masked language modeling
- Discriminator task: *Replaced token detection*

→ Predict for each token, whether it is "original" or produced by G

- ELECTRA learns from *all* of the tokens (not just from a small portion of 15%, like e.g. BERT)

ELECTRA – ARCHITECTURE

Joint pre-training:



▸ Source: Clark et al, 2020

- G and D are (Transformer) encoders which are trained jointly
- G replaces `[MASK]`s in an input sequence
  $\rightarrow$ Passes corrupted input sequence $\vec{x}^{corrupt}$ to D

ELECTRA – TRAINING DETAILS

Joint pre-training:

- Generation of samples:

$$m_i \sim \text{unif}\{1, n\} \text{ for } i = 1 \text{ to } k \qquad \boldsymbol{x}^{\text{masked}} = \text{REPLACE}(\boldsymbol{x}, \boldsymbol{m}, \texttt{[MASK]})$$
$$\hat{x}_i \sim p_G(x_i | \boldsymbol{x}^{\text{masked}}) \text{ for } i \in \boldsymbol{m} \qquad \boldsymbol{x}^{\text{corrupt}} = \text{REPLACE}(\boldsymbol{x}, \boldsymbol{m}, \hat{\boldsymbol{x}})$$

with approx. 15% of the tokens masked out (via choice of k)

- D predicts whether x_t, $t \in 1, \ldots, T$ is "*real*" or generated by G
  - Softmax output layer for G (probability distr. over all words)
  - Sigmoid output layer for D (Binary classification real vs. generated)

ELECTRA – TRAINING DETAILS

Using the masked & corrupted input sequences, the (joint) loss can be written down as follows:

Loss functions:

$$\mathcal{L}_{\text{MLM}}(\boldsymbol{x}, \theta_G) = \mathbb{E}\left(\sum_{i \in \boldsymbol{m}} -\log p_G(x_i | \boldsymbol{x}^{\text{masked}})\right)$$

$$\mathcal{L}_{\text{Disc}}(\boldsymbol{x}, \theta_D) = \mathbb{E}\left(\sum_{t=1}^{n} -\mathbb{1}(x_t^{\text{corrupt}} = x_t) \log D(\boldsymbol{x}^{\text{corrupt}}, t) - \mathbb{1}(x_t^{\text{corrupt}} \neq x_t) \log(1 - D(\boldsymbol{x}^{\text{corrupt}}, t))\right)$$

Combined:

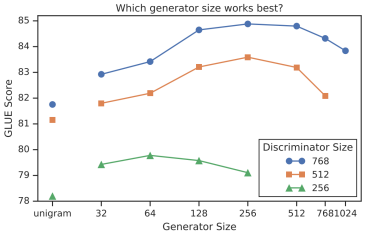
$$\min_{\theta_G, \theta_D} \sum_{\boldsymbol{x} \in \mathcal{X}} \mathcal{L}_{\text{MLM}}(\boldsymbol{x}, \theta_G) + \lambda \mathcal{L}_{\text{Disc}}(\boldsymbol{x}, \theta_D)$$

with λ set to 50, since the discriminator's loss is typically much lower than the geneator's.

ELECTRA – TRAINING DETAILS

Generator size:

- Same size of G and D:
  - Twice the compute per training step + too challenging for D
- Smaller Generators are preferable ($1/4 - 1/2$ the size of D)

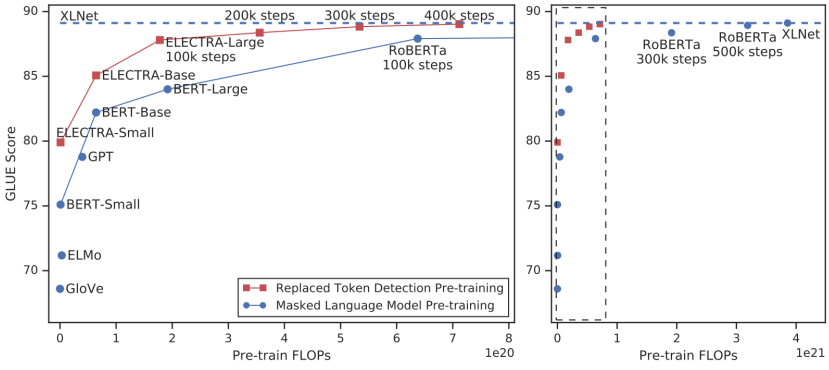


▸ Source: Clark et al, 2020

Weight sharing (experimental):

- Same size of G and D: All weights can be tied
- G smaller than D: Share token & positional embeddings

ELECTRA – MODEL COMPARISON



▸ Source: Clark et al, 2020

Note: Different batch sizes (2k vs. 8k) for ELECTRA vs. RoBERTa/XLNet explain why same number of steps lead to approx. 1/4 of the compute for ELECTRA.

ELECTRA – SOTA PERFORMANCE

Performance differences vs. BERT/RoBERTa (GLUE dev set):

Model	Train FLOPs	Params	CoLA	SST	MRPC	STS	QQP	MNLI	QNLI	RTE	Avg.
BERT	1.9e20 (0.27x)	335M	60.6	93.2	88.0	90.0	91.3	86.6	92.3	70.4	84.0
RoBERTa-100K	6.4e20 (0.90x)	356M	66.1	95.6	**91.4**	92.2	92.0	89.3	94.0	82.7	87.9
RoBERTa-500K	3.2e21 (4.5x)	356M	68.0	96.4	90.9	92.1	92.2	90.2	94.7	86.6	88.9
XLNet	3.9e21 (5.4x)	360M	69.0	**97.0**	90.8	92.2	92.3	90.8	94.9	85.9	89.1
BERT (ours)	7.1e20 (1x)	335M	67.0	95.9	89.1	91.2	91.5	89.6	93.5	79.5	87.2
ELECTRA-400K	7.1e20 (1x)	335M	**69.3**	96.0	90.6	92.1	**92.4**	90.5	94.5	86.8	89.0
ELECTRA-1.75M	3.1e21 (4.4x)	335M	69.1	96.9	90.8	**92.6**	**92.4**	**90.9**	**95.0**	**88.0**	**89.5**

▸ Source: Clark et al, 2020

SOTA performance (GLUE test set):

Model	Train FLOPs	CoLA	SST	MRPC	STS	QQP	MNLI	QNLI	RTE	WNLI	Avg.*	Score
BERT	1.9e20 (0.06x)	60.5	94.9	85.4	86.5	89.3	86.7	92.7	70.1	65.1	79.8	80.5
RoBERTa	3.2e21 (1.02x)	67.8	96.7	89.8	91.9	90.2	90.8	95.4	88.2	89.0	88.1	88.1
ALBERT	3.1e22 (10x)	69.1	**97.1**	**91.2**	92.0	90.5	**91.3**	–	89.2	91.8	89.0	–
XLNet	3.9e21 (1.26x)	70.2	**97.1**	90.5	**92.6**	90.4	90.9	–	88.5	**92.5**	89.1	–
ELECTRA	3.1e21 (1x)	**71.7**	**97.1**	90.7	92.5	**90.8**	**91.3**	**95.8**	**89.8**	**92.5**	**89.5**	**89.4**

* Avg. excluding QNLI to ensure comparability

▸ Source: Clark et al, 2020

DIFFERENT PRE-TRAINING REGIMES (2)

Autoregressive (AR) language modeling

- Factorizes likelihood to

$$p(\mathrm{x}) = \prod_{t=1}^{T} p(x_t | \mathrm{x}_{<t})$$

- Only uni-directional (backward factorization also possible)

vs. (Denoising) Autoencoding (AE)

- Reconstruct masked tokens $\bar{\mathrm{x}}$ from corrupted sequence $\hat{\mathrm{x}}$:

$$p(\bar{\mathrm{x}} | \hat{\mathrm{x}}) = \prod_{t=1}^{T} m_t \cdot p(x_t | \hat{\mathrm{x}}), \text{with } m_t \text{ as masking indicator}$$

- Possible "*corruptions*" (besides MLM):
  → Replacing words with predictions (cf. ELECTRA)
  → Shuffling tokens or dropping them

DIFFERENT PRE-TRAINING REGIMES (2)

Conceptual Differences

- **AR:** No corruption of input sequences required
- "Causal" structure in AR approach (sometimes needed)
- **AE:** Assumes independence between corrupted tokens

  Question: Why?
- AR approach only conditions on left side context

  $\rightarrow$ No bidirectionality possible

PERMUTATION LANGUAGE MODELING (PLM)

XLNet ▸ Yang et al., 2019

- Alternative objective function
- Solves the pretrain-finetune discrepancy

→ No artificial `[MASK]` token is introduced

- Allows for bidirectionality while keeping an AR objective

→ Best of both worlds

- Consists of two "*streams*" of the Attention mechanism
  - Content-stream attention
  - Query-stream attention

PERMUTATION LANGUAGE MODELING (PLM)

Manipulating the factorization order

- Consider permutations z of the index sequence $[1, 2, \ldots, T]$

$\rightarrow$ Used to permute the factorization order, *not* the sequence order.

- Original order of the sequence is retained via positional encodings
- PLM objective (with $\mathcal{Z}_T$ as set of all possible permutations):

$$\max_{\theta} \quad \mathbb{E}_{\mathsf{z} \sim \mathcal{Z}_T} \left[\sum_{t=1}^{T} \log p_\theta(x_{z_t} | \mathsf{x}_{\mathsf{z}_{<t}}) \right]$$

PLM – CONTENT- VS. QUERY-STREAM

Content-stream

- "Ordinary" Self-Attention (with special attention masks)
  $\rightarrow$ Attentions masks depend on the factorization order
- Information about the position in the sequence is lost,
  see ▸ Example in A.1 (Yang et al., 2019)
- Sets of queries (Q), keys (K) and values (V) from content stream*
- Yields a *content embedding* denoted as $h_\theta(\mathrm{x}_{\mathrm{z}_{\leq t}})$

*For a nice visual disentanglement, see ▸ Figures in A.7 (Yang et al., 2019)

PLM – CONTENT- VS. QUERY-STREAM

Content-stream

A.1 A Concrete Example of How Standard LM Parameterization Fails

In this section, we provide a concrete example to show how the standard language model parameterization fails under the permutation objective, as discussed in Section 2.3. Specifically, let's consider two different permutations $\mathbf{z}^{(1)}$ and $\mathbf{z}^{(2)}$ satisfying the following relationship

$$\mathbf{z}^{(1)}_{<t} = \mathbf{z}^{(2)}_{<t} = \mathbf{z}_{<t} \quad \text{but} \quad z^{(1)}_t = i \neq j = z^{(2)}_t.$$

Then, substituting the two permutations respectively into the naive parameterization, we have

$$\underbrace{p_\theta(X_i = x \mid \mathbf{x}_{\mathbf{z}_{<t}})}_{z^{(1)}_t = i,\, \mathbf{z}^{(1)}_{<t} = \mathbf{z}_{<t}} = \underbrace{p_\theta(X_j = x \mid \mathbf{x}_{\mathbf{z}_{<t}})}_{z^{(1)}_t = j,\, \mathbf{z}^{(2)}_{<t} = \mathbf{z}_{<t}} = \frac{\exp\left(e(x)^\top h(\mathbf{x}_{\mathbf{z}_{<t}})\right)}{\sum_{x'} \exp\left(e(x')^\top h(\mathbf{x}_{\mathbf{z}_{<t}})\right)}.$$

Effectively, two different target positions i and j share exactly the same model prediction. However, the ground-truth distribution of two positions should certainly be different.

▸ Source: Yang et al., 2019

PLM – CONTENT- VS. QUERY-STREAM

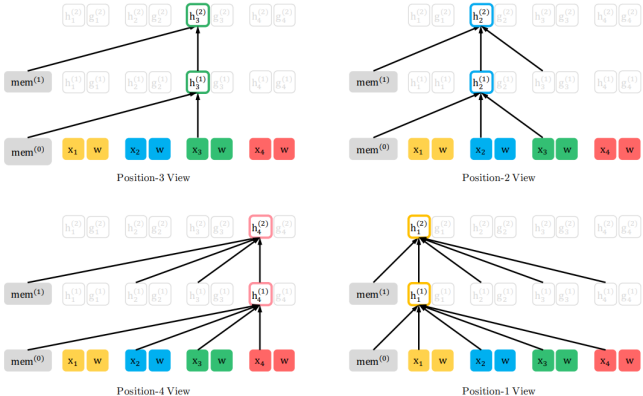
Query-stream

- Access to context through content-stream, but no access to the content of the current position (only location information)
- Q from the query stream, K and V from the content stream*
- Yields a *query embedding* denoted as $g_\theta(\mathrm{x}_{\mathrm{z}_{<t}}, z_t)$

*For a nice visual disentanglement, see ▸ Figures in A.7 (Yang et al., 2019)

PLM – CONTENT- VS. QUERY-STREAM

Content-stream

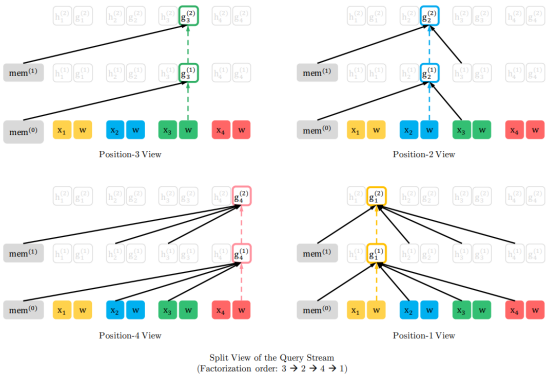


Split View of the Content Stream
(Factorization order: 3 → 2 → 4 → 1)

▸ Source: Yang et al., 2019

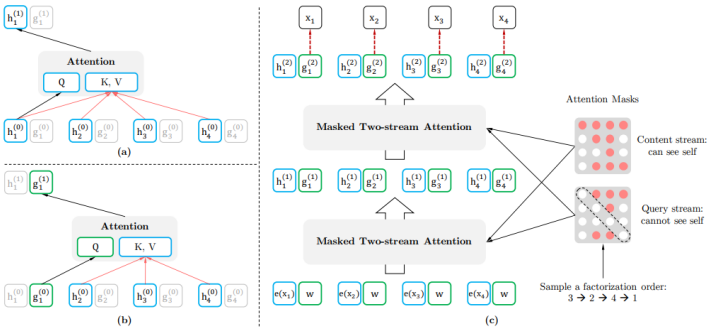
PLM – CONTENT- VS. QUERY-STREAM

Query-stream



▸ Source: Yang et al., 2019

PLM – CONTENT- VS. QUERY-STREAM



(a) content-stream; (b) query-stream; (c) whole model

▸ Source: Yang et al., 2019

XLNET – MODEL INPUT

Generation of samples:

- Randomly sample two sentences and use concatenation* as input

[CLS]	The	fox	is	quick	.	[SEP]	
It	jumps	over	the	lazy	dog	.	[SEP]

*Nevertheless: XLNet does *not* use the NSP objective

Additional encodings:

- *Relative* segment encodings:
  - BERT adds absolute segment embeddings (E_A & E_B)
  - XLNet uses relative encodings ($\vec{s}_+$ & $\vec{s}_-$)
- *Relative* Positional encodings:
  - BERT encodes information about the absolute position ($E_0, E_1, E_2, \ldots$)
  - XLNet uses relative encodings (R_{i-j})

XLNET – SPECIAL REMARKS

- **Partial Prediction:** Only predict the last tokens in a factoriztion order (reduces optimization difficulty)

$$\max_{\theta} \quad \mathbb{E}_{\mathbf{z}\sim\mathcal{Z}_T}\left[\sum_{t=c+1}^{|\mathbf{z}|} \log p_\theta(x_{\mathbf{z}_t}|\mathbf{x}_{\mathbf{z}_{<t}})\right], \quad \text{with } c \text{ as cutting point}$$

- **Segment recurrence mechanism:** Allow for learning extended contexts in Transformer-based architectures, see ▸ Dai et al. (2019)
- **No independence assumption:**

$$\mathcal{J}_{\text{BERT}} = \log p(\text{New} \mid \text{is a city}) + \log p(\text{York} \mid \text{is a city}),$$

$$\mathcal{J}_{\text{XLNet}} = \log p(\text{New} \mid \text{is a city}) + \log p(\text{York} \mid \text{New}, \text{is a city})$$

Prediction of [New, York] given the factorization order [is, a, city, New, York]

▸ Source: Yang et al., 2019

XLNET – SOTA PERFORMANCE

Performance differences to BERT:

Model	SQuAD1.1	SQuAD2.0	RACE	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	STS-B
BERT-Large (Best of 3)	86.7/92.8	82.8/85.5	75.1	87.3	93.0	91.4	74.0	94.0	88.7	63.7	90.2
XLNet-Large-wikibooks	88.2/94.0	85.1/87.8	77.4	88.4	93.9	91.8	81.2	94.4	90.0	65.2	91.1

Table 1: Fair comparison with BERT. All models are trained using the same data and hyperparameters as in BERT. We use the best of 3 BERT variants for comparison; i.e., the original BERT, BERT with whole word masking, and BERT without next sentence prediction.

▸ Source: Yang et al., 2019

SOTA performance:

Model	MNLI	QNLI	QQP	RTE	SST-2	MRPC	CoLA	STS-B	WNLI
Single-task single models on dev									
BERT [2]	86.6/-	92.3	91.3	70.4	93.2	88.0	60.6	90.0	-
RoBERTa [21]	90.2/90.2	94.7	92.2	**86.6**	96.4	**90.9**	68.0	92.4	-
XLNet	**90.8/90.8**	**94.9**	**92.3**	85.9	**97.0**	90.8	**69.0**	**92.5**	-
Multi-task ensembles on test (from leaderboard as of Oct 28, 2019)									
MT-DNN* [20]	87.9/87.4	96.0	89.9	86.3	96.5	92.7	68.4	91.1	89.0
RoBERTa* [21]	90.8/90.2	98.9	90.2	88.2	96.7	92.3	67.8	92.2	89.0
XLNet*	**90.9/90.9**†	**99.0**†	**90.4**†	**88.5**	**97.1**†	**92.9**	**70.2**	**93.0**	**92.5**

Table 5: Results on GLUE. ∗ indicates using ensembles, and † denotes single-task results in a multi-task row. All dev results are the median of 10 runs. The upper section shows direct comparison on dev data and the lower section shows comparison with state-of-the-art results on the public leaderboard.

▸ Source: Yang et al., 2019

Tasks as Text-to-Text Problem

RECAP: BERT, ROBERTA, ETC.

- Transformer encoder
- Training: Masked language modeling (or similar)
- BERT* learns an enormous amount of knowledge about language and the world through MLM training on large corpora
- Application: fine-tune on a particular task
- Great performance!
- **Question:** What's not to like?

*In what follows we will use BERT as a representative for this class of language models, i.e. models with a custom classification layer of each task, and only talk about BERT – but the discussion includes RoBERTa, Albert, XLNet etc.

PROBLEMS WITH BERT (1)

- You need a different model for each task.

  (Because BERT is differently fine-tuned for each task.)

  $\rightarrow$ Not realistic in many real deployment scenarios, e.g., on mobile devices.
- *Human learning:* We arguably have a single model that solves all tasks!
- **Question:** Is there a framework that allows us to create a single model that solves all tasks?

PROBLEMS WITH BERT (2)

- Two training modes: first (MLM) pretraining, then fine-tuning.
- Fine-tuning is supervised learning, i.e., learning from labeled examples.
- Arguably, learning from labeled examples is untypical for human learning.
- You never learn a task solely by being presented a bunch of examples, without explanation.
- Instead, in human learning, there is almost always a task description.

PROBLEMS WITH BERT (2)

- Example: *How to boil an egg.*
  - "Place eggs in the bottom of a saucepan."
  - "Fill the pan with cold water."
  - "Etc."
- Notice that this is not an example but a *description* of the task
- **Question:** Is there a framework that allows us to leverage task descriptions?

PROBLEMS WITH BERT (3)

- BERT has great performance, but ...
- ...only if the training set is large, generally 1000s of examples
- This is completely different from human learning!
- We do use examples in learning, but in most cases, only a few

PROBLEMS WITH BERT (3)

- Example: Maybe the person teaching you how to boil an egg will show you how to do it *one or two times*
- But probably not 10 times
- Definitely not a 1000 times
- More practical concern: it's very expensive to label 1000s of examples for each task (there are many tasks).
- **Question:** Is there a framework that allows us to learn from just a small number of examples?
- This is called few-shot learning.

PROBLEMS WITH BERT (4)

- More subtle aspect of the same problem (i.e., large training sets):
  $\rightarrow$ **Overfitting**
- Even though performance looks good on standard train/dev/test splits, the deviation between the training set and the data actually encountered in real application can be large
- So our benchmarks often overestimate what performance would be in reality

PROBLEMS WITH BERT – SUMMARY

- No true multitask learning
- Differs from the way humans learn
- No possibility/necessity to make use of a description of the task
- Overfitting

REVISITING TEXT-TO-TEXT TASKS

Example: Machine Translation

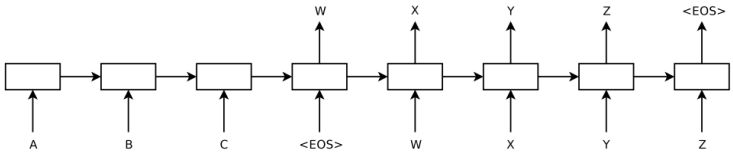
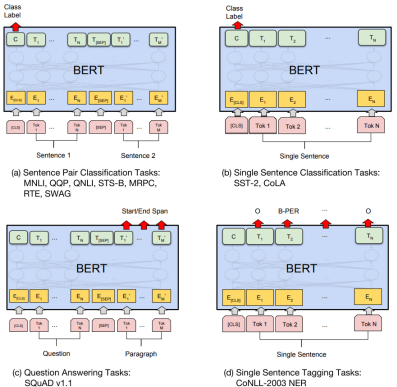


Figure 1: Our model reads an input sentence "ABC" and produces "WXYZ" as the output sentence. The model stops making predictions after outputting the end-of-sentence token. Note that the LSTM reads the input sentence in reverse, because doing so introduces many short term dependencies in the data that make the optimization problem much easier.

▸ Source: Sutskever et al., 2014

- *Input:* Text in source language
- *Output:* Text in target language

REVISITING FINE-TUNING OF BERT



▸ Source: Devlin et al., 2019

- *First row:* (sequence-level) classification
- *Second row:* sequence tagging = token-level classification

FINE-TUNING + TEXT-TO-TEXT?

- **Question:** How does BERT know which task it is performing?
  - It doesn't!
  - BERT just learns associations between text features and class distributions
  - No meaningful internal representation of what "class 3" actually *means*
  - *Input*: text; *Output*: `[0 0 1 0 0 0]`
- The fact that BERT does not "understand" what task it is actually performing is the reason why one copy is needed per task

FINE-TUNING + TEXT-TO-TEXT?

- **Question:** How could we "inform" our model about the task?
  - Model takes text as an input and can process.
  - Why not describe the task in natural language?
  - *Assumption:*
    Given the model sees 1000s of (inputs; ouput) pairs like ..

("`<task description>: <input sample>`" ; "`<class name>`")

  - .. it will learn to associate ..
    → .. task descriptions to a set of class names
    → .. text features to class name distributions

FINE-TUNING + TEXT-TO-TEXT?

- **Question:** How does a "good" task description look like?
  - Remember: The idea is to have a model that learns to associate the task descriptions with a set of class names
  - *Class names:*
    $\rightarrow$ Could be pretty new/exotic terms the model is not familiar with after pre-training
    $\rightarrow$ It will learn to output them during fine-tuning when "triggered" accordingly (i.e. with the right task description)
  - *Task descriptions:*
    $\rightarrow$ Might be possible that the the model already saw some task descriptions during pre-training and has a meaningful representation for them.
    $\rightarrow$ On the other hand: Others might also be pretty new/exotic.

FINE-TUNING + TEXT-TO-TEXT?

For sentiment classification, I can just provide it with 1000s of training samples like

(“<sentiment classification>: <input sample>“ ;
“<positive/negative>“)

The model will ..

- .. utilize already present knowledge about the task description.
- .. learn to associate the task description precisely with this label set

FINE-TUNING + TEXT-TO-TEXT?

- **Question:** If I want the model to do sentiment classification, can I also just provide it with 1000s of training samples like

  (“`<khaleesi>: <input sample>`“ ; “`<positive/negative>`“)

  ?

  (*khaleesi*[1] just used as some random fantasy word here)

FINE-TUNING + TEXT-TO-TEXT?

- **Question:** If I want the model to do sentiment classification, can I also just provide it with 1000s of training samples like

  (“`<khaleesi>: <input sample>`“ ; “`<positive/negative>`“)

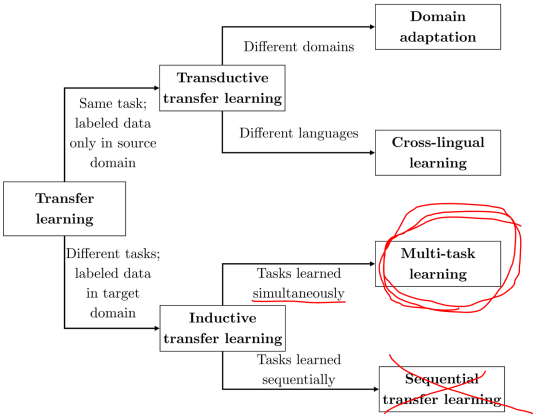
  ?

  (*khaleesi*[1] just used as some random fantasy word here)
- *Intuition:* Yes! It basically does not matter, **as long as the model is fine-tuned on this data**[2]
- Fine-tuning important for the model to pick up on this “signal“ and output according text (corresponding the appropriate classes)

[1] Khaleesi is a Dothraki title referring to the wife of the khal (cf. Game of Thrones).

[2] Of course, using “good“ task descriptions (as shown on the previous slide) that are also meaningful to human readers make more sense as the model might learn quicker because they diverge less from the pre-training distribution.

MULTI-TASK LEARNING?



▸ Taxonomy of transfer learning (Source: Ruder, 2019)

MULTI-TASK LEARNING?

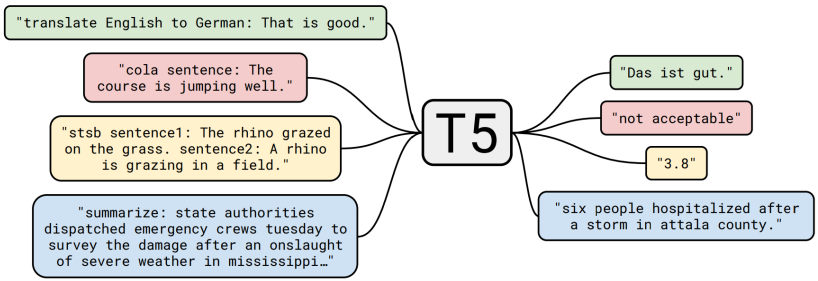
- **Question:** If the model can learn to link task descriptions to label sets, can it learn to perform multiple tasks at once?
- Yes, through fine tuning each task description[1] will be linked to its label set:
  - `<task description a>` → `{positive, negative}`
  - `<task description b>` → `{world, sports, business, sci/tech}` ▸ Example from AG News
  - `<task description c>` → `{spam, no spam}`
  - `<task description d>` → `{entailment, contradiction, neutral}`

[1] `<task description a>` (and so on) are just placeholders here.

MULTI-TASK LEARNING?

- **Question:** Can we have one single model for both different classification tasks **and** generation tasks?
  - Yes, we just have to design suitable task descriptions
  - Examples for *generative* descriptions ▸ Raffel et al., 2020
    - `translate English to German: <input sample>`
    - `summarize: <input sample>`
    - `answer question: <input sample>`
  - Examples for *classification* descriptions ▸ Raffel et al., 2020
    - `binary classification: <input sample>`
    - `predict sentiment: <input sample>`

TEXT-TO-TEXT TASKS

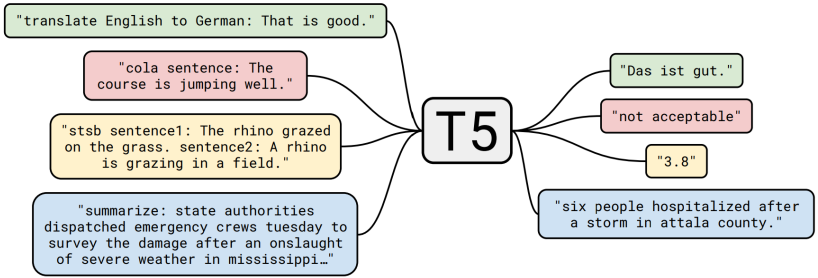


▸ Source: Raffel et al., 2020

- **Important note:** What we talked about as `<task description>` until now is commonly referred to as `<task prefix>`

Text-to-Text Transfer Transformer

REVISITING TEXT-TO-TEXT TASKS



▸ Source: Raffel et al., 2020

Ingredients:

- `<task prefix>` for each task
  → Concatenation with the input
- Output label / score formatted as string

TEXT-TO-TEXT TRANSFER TRANSFORMER

Short summary:

- In short: T5 (bc of the five Ts)
- A complete encoder-decoder Transformer architecture
  Question: Why is this important?
- Relative positional embeddings
- All tasks reformulated as text-to-text tasks
  $\rightarrow$ Probably the most important innovation of this work
- From BERT-size $\times 2$ (since enc-dec) up to 11B parameters
- Paradigm shift from *Sequential Transfer Learning* towards true *Multi-Task Learning*

▸ Nice Animation (similar to figure before)

INPUT AND OUTPUT FORMAT

Input Side:

- SentencePiece framework w/ WordPiece tokens
- Add task-specific (text) prefix to the original input
- Choice of the prefix: Hyperparameter!!

→ Changing this had limited impact

→ No extensive experiments performed by the authors

Output Side:

- Output as a word or a piece of text (also numerical similarity scores)
- If output not present in set of potential alternatives, prediction considered as wrong (many other possible ways; this is not trivial)

TASK PREFIX VS. PROMPTING

Adding task-specific (text) prefix:

- Add task-specific (text) prefix to the original input
- Model is fine-tuned on samples prepended with this prefix

→ It learns to recognize what it is expected to do when encountering a specific prefix at test time

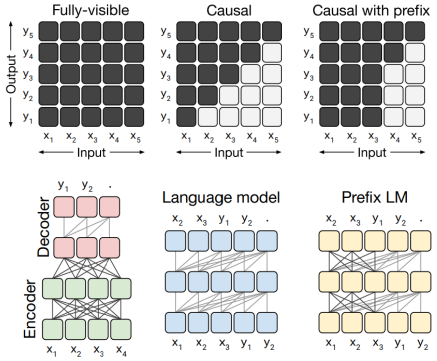
→ Probably because of this: limited impact found by the authors

Prompting:

- Refers to just querying a trained w/o fine-tuning it (cf. next chapter)
- Paradigm of Few-/Zero-Shot Learning
- This is found to have a *huge* impact on model performance

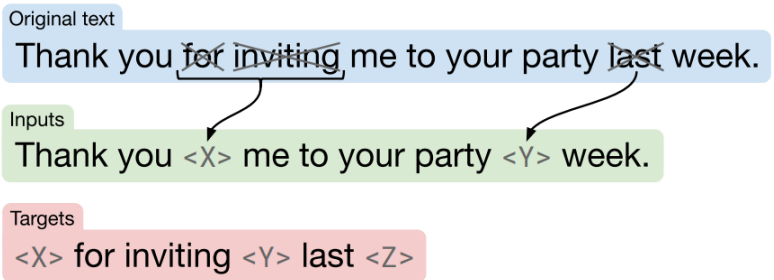
ARCHITECTURAL DIFFERENCES

Attention patterns (top) and Schematics of considered architectures (bottom)



Source: Raffel et al., 2020

PRE-TRAINING OBJECTIVE



▸ Source: Raffel et al., 2020

1. Mark spans in input sequence for corruptions
2. Replace tokens with sentinel tokens
3. Predict sentinel tokens and replaced tokens

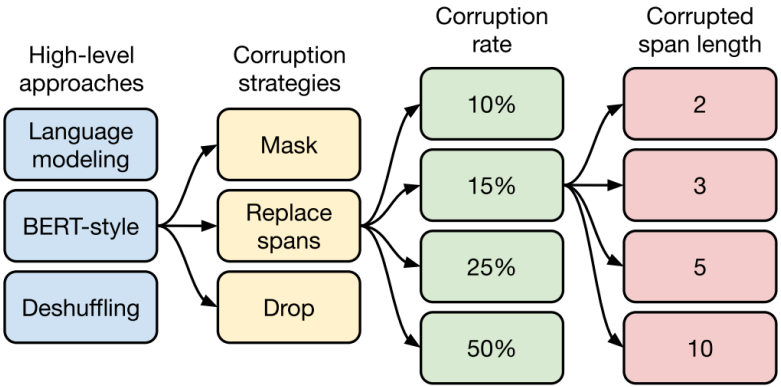
PRE-TRAINING OBJECTIVES

- Authors experimented with different objectives
- Most of them rely in some way on MLM

Objective	Inputs	Targets
Prefix language modeling	Thank you for inviting	me to your party last week .
BERT-style Devlin et al. (2018)	Thank you <M> <M> me to your party apple week .	*(original text)*
Deshuffling	party me for your to . last fun you inviting week Thank	*(original text)*
MASS-style Song et al. (2019)	Thank you <M> <M> me to your party <M> week .	*(original text)*
I.i.d. noise, replace spans	Thank you <X> me to your party <Y> week .	<X> for inviting <Y> last <Z>
I.i.d. noise, drop tokens	Thank you me to your party week .	for inviting last
Random spans	Thank you <X> to <Y> week .	<X> for inviting me <Y> your party last <Z>

▸ Source: Raffel et al., 2020

PRE-TRAINING OBJECTIVES



▸ Source: Raffel et al., 2020

THE COLOSSAL CLEAN CRAWLED CORPUS (C4)

- Effort to measure the effect of quality, characteristics & size of the pre-training resources
- Common Crawl as basis, careful cleaning and filtering for English language
- Orders of magnitude larger (750GB) compared to commonly used corpora

THE COLOSSAL CLEAN CRAWLED CORPUS (C4)

Experiments (with respect to C4)

Data set	Size	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ C4	745GB	83.28	**19.24**	80.88	71.36	**26.98**	**39.82**	**27.65**
C4, unfiltered	6.1TB	81.46	19.14	78.78	68.04	26.55	39.34	27.21
RealNews-like	35GB	**83.83**	**19.23**	80.39	72.38	**26.75**	**39.90**	**27.48**
WebText-like	17GB	**84.03**	**19.31**	**81.42**	71.40	**26.80**	**39.74**	**27.59**
Wikipedia	16GB	81.85	**19.31**	81.29	68.01	**26.94**	39.69	**27.67**
Wikipedia + TBC	20GB	83.65	**19.28**	**82.08**	**73.24**	**26.77**	39.63	**27.57**

Number of tokens	Repeats	GLUE	CNNDM	SQuAD	SGLUE	EnDe	EnFr	EnRo
★ Full data set	0	**83.28**	**19.24**	**80.88**	**71.36**	**26.98**	**39.82**	**27.65**
2^{29}	64	**82.87**	**19.19**	**80.97**	**72.03**	**26.83**	**39.74**	**27.63**
2^{27}	256	82.62	**19.20**	79.78	69.97	**27.02**	**39.71**	27.33
2^{25}	1,024	79.55	18.57	76.27	64.76	26.38	39.56	26.80
2^{23}	4,096	76.34	18.33	70.92	59.29	26.37	38.84	25.81

▸ Source: Raffel et al., 2020

T5 - EXHAUSTIVE EXPERIMENTS

Performed experiments with respect to ..

- *.. architecture, size & objective*

→ enc, dec, enc-dec

→ Between 60M and 11B parameters

- *.. details of the Denoising objective (which was found to work best)*
- *.. fine-tuning methods*

→ Adapter layers

→ Gradual Unfreezing (cf. ULMFiT)

- *.. Multi-task learning strategies*

→ Examples-proportional mixing

→ Temperature-scaled mixing

BENCHMARK RESULTS

Model	GLUE Average	CoLA Matthew's	SST-2 Accuracy	MRPC F1	MRPC Accuracy	STS-B Pearson	STS-B Spearman
Previous best	89.4[a]	69.2[b]	97.1[a]	**93.6**[b]	**91.5**[b]	92.7[b]	92.3[b]
T5-Small	77.4	41.0	91.8	89.7	86.6	85.6	85.0
T5-Base	82.7	51.1	95.2	90.7	87.5	89.4	88.6
T5-Large	86.4	61.2	96.3	92.4	89.9	89.9	89.2
T5-3B	88.5	67.1	97.4	92.5	90.0	90.6	89.8
T5-11B	**90.3**	**71.6**	**97.5**	92.8	90.4	**93.1**	**92.8**

Model	QQP F1	QQP Accuracy	MNLI-m Accuracy	MNLI-mm Accuracy	QNLI Accuracy	RTE Accuracy	WNLI Accuracy
Previous best	74.8[c]	**90.7**[b]	91.3[a]	91.0[a]	**99.2**[a]	89.2[a]	91.8[a]
T5-Small	70.0	88.0	82.4	82.3	90.3	69.9	69.2
T5-Base	72.6	89.4	87.1	86.2	93.7	80.1	78.8
T5-Large	73.9	89.9	89.9	89.6	94.8	87.2	85.6
T5-3B	74.4	89.7	91.4	91.2	96.3	91.1	89.7
T5-11B	**75.1**	90.6	**92.2**	**91.9**	96.9	**92.8**	**94.5**

▸ Source: Raffel et al., 2020

BENCHMARK RESULTS

Model	SQuAD EM	SQuAD F1	SuperGLUE Average	BoolQ Accuracy	CB F1	CB Accuracy	COPA Accuracy
Previous best	90.1[a]	95.5[a]	84.6[d]	87.1[d]	90.5[d]	95.2[d]	90.6[d]
T5-Small	79.10	87.24	63.3	76.4	56.9	81.6	46.0
T5-Base	85.44	92.08	76.2	81.4	86.2	94.0	71.2
T5-Large	86.66	93.79	82.3	85.4	91.6	94.8	83.4
T5-3B	88.53	94.95	86.4	89.9	90.3	94.4	92.0
T5-11B	**91.26**	**96.22**	**88.9**	**91.2**	**93.9**	**96.8**	**94.8**

Model	MultiRC F1a	MultiRC EM	ReCoRD F1	ReCoRD Accuracy	RTE Accuracy	WiC Accuracy	WSC Accuracy
Previous best	84.4[d]	52.5[d]	90.6[d]	90.0[d]	88.2[d]	69.9[d]	89.0[d]
T5-Small	69.3	26.3	56.3	55.4	73.3	66.9	70.5
T5-Base	79.7	43.1	75.0	74.2	81.5	68.3	80.8
T5-Large	83.3	50.7	86.8	85.9	87.8	69.3	86.3
T5-3B	86.8	58.3	91.2	90.4	90.7	72.1	90.4
T5-11B	**88.1**	**63.3**	**94.1**	**93.4**	**92.5**	**76.9**	**93.8**

▸ Source: Raffel et al., 2020

BENCHMARK RESULTS

Model	WMT EnDe BLEU	WMT EnFr BLEU	WMT EnRo BLEU	CNN/DM ROUGE-1	CNN/DM ROUGE-2	CNN/DM ROUGE-L
Previous best	**33.8**[e]	**43.8**[e]	**38.5**[f]	43.47[g]	20.30[g]	40.63[g]
T5-Small	26.7	36.0	26.8	41.12	19.56	38.35
T5-Base	30.9	41.2	28.0	42.05	20.34	39.40
T5-Large	32.0	41.5	28.1	42.50	20.68	39.75
T5-3B	31.8	42.6	28.2	42.72	21.02	39.94
T5-11B	32.1	43.4	28.1	**43.52**	**21.55**	**40.69**

▸ Source: Raffel et al., 2020

T5 - EXHAUSTIVE EXPERIMENTS

Conclusions

- Encoder-decoder architecture works best in this "text-to-text" setting
- More data, larger models & ensembling all boost the performance
  - Larger models trained for fewer steps better than smaller models on more data
  - Ensembling: Using same base pre-trained models worse than complete separate model ensembles
- Different denoising objectives work similarly well
- Updating *all* model parameters during fine-tuning works best (but expensive)

DIFFERENT T5 VERSIONS

Similar to BERT T5 has different versions, which all have a different parameter count ▸ Raffel et al., 2020

Model	Parameters	n_{layers}	d_{model}	d_{FFN}	d_{qkv}	n_{heads}
small	60M	6	512	2048	64	8
base	220M	12	768	3072	64	12
large	770M	24	1024	4096	64	16
3B	3B	24	1024	16384	128	32
11B	11B	24	1024	65536	128	128

- For T5 small, base and large we have the following parameter count per encoder/decoder layer:
  - $N_{Encoder} = 12 \cdot d_{model}^2$
  - $N_{Decoder} = 16 \cdot d_{model}^2$
- *For T5 3B and 11B $N_{Encoder}$ and $N_{Decoder}$ are different!*

APPROXIMATE PARAMETER COUNT (1)

The general formula is:

$$N_{Total} = n_{layers} \cdot (N_{Encoder} + N_{Decoder}) + N_{Embedding}$$

- Let $V = 32000$ be the size of the vocabulary, we then have:

$$N_{Embedding} = 32000 \times d_{model}$$

- **T5 small**:

$$N_{small} = 6 \cdot (12 \cdot 512^2 + 16 \cdot 512^2) + 32000 \times 512 = 60,424,192$$

- **T5 base**:

$$N_{base} = 12 \cdot (12 \cdot 768^2 + 16 \cdot 768^2) + 32000 \times 768 = 222,756,864$$

- **T5 large**:

$$N_{large} = 24 \cdot (12 \cdot 1024^2 + 16 \cdot 1024^2) + 32000 \times 1024 = 737,411,072$$

REVISITING: MHA AND FFN (1)

- In the transformer chapter we derived the FFN parameter count to be $8 \cdot \boldsymbol{d_{model}}$
- We also derived the MHA parameter count to be $4 \cdot \boldsymbol{d_{model}^2}$
- But this is only the case if we choose $d_{FFN} = 4 \cdot d_{model}$ and $d_{qkv} = \frac{d_{model}}{n_{heads}}$, with d_{FFN} being the hidden dimension of the FFN and d_{qkv} being the embedding dimension of each head in MHA
- For the two bigger model variants we have the following relations:
  - **T5 3B**: $d_{FFN} = 16 \cdot d_{model}$ and $d_{qkv} \neq \frac{d_{model}}{n_{heads}}$
  - **T5 11B**: $d_{FFN} = 64 \cdot d_{model}$ and $d_{qkv} \neq \frac{d_{model}}{n_{heads}}$
- Thats why we can't use the same formulas as for the three previous variants

REVISITING: MHA AND FFN (2)

- In MHA $n_{heads} \cdot d_{qkv}$ is no longer d_{model}:

$$\begin{aligned} N_{attention} &= n_{heads} \cdot d_{qkv} \times d_{model} + (d_{model} \times d_{qkv}) \times n_{heads} \cdot 3 \\ &= 4 \cdot n_{heads} \cdot d_{qkv} \times d_{model}; \quad \text{instead of:} \quad 4 \cdot d_{model}^2 \end{aligned}$$

- For the FFN d_{FFN} is no longer $4 \cdot d_{model}$:

$$\begin{aligned} N_{FFN} &= d_{model} \times d_{FFN} + d_{FFN} \times d_{model} \\ &= 2 \cdot d_{model} \times d_{FFN}; \quad \text{instead of:} \quad 8 \cdot d_{model}^2 \end{aligned}$$

- For the Encoder and Decoder that means:
  - $N_{Encoder} = 4 \cdot n_{heads} \cdot d_{qkv} \times d_{model} + 2 \cdot d_{model} \times d_{FFN}$
  - $N_{Decoder} = 2 \cdot 4 \cdot n_{heads} \cdot d_{qkv} \times d_{model} + 2 \cdot d_{model} \times d_{FFN}$

APPROXIMATE PARAMETER COUNT (2)

The general formula is still:

$$N_{Total} = n_{layers} \cdot (N_{Encoder} + N_{Decoder}) + N_{Embedding}$$

- **T5 3B**:

$$\begin{aligned} N_{3B} &= 24 \cdot ([4 \cdot 32 \cdot 128 \times 1024 + 2 \cdot 1024 \times 16384] \\ &\quad + [2 \cdot 4 \cdot 32 \cdot 128 \times 1024 + 2 \cdot 1024 \times 16384]) + 32000 \cdot 1024 \\ &= 2,851,340,288 \approx 3B \end{aligned}$$

- **T5 11B**:

$$\begin{aligned} N_{11B} &= 24 \cdot ([4 \cdot 128 \cdot 128 \times 1024 + 2 \cdot 1024 \times 65536] \\ &\quad + [2 \cdot 4 \cdot 128 \cdot 128 \times 1024 + 2 \cdot 1024 \times 65536]) + 32000 \cdot 1024 \\ &= 11,307,057,152 \approx 11B \end{aligned}$$

License & Attribution

LICENSE & ATTRIBUTION

- **Single source.** All content is taken from **Deep Learning for NLP (DL4NLP)**, SLDS, LMU Munich, chapters 4 (*BERT*), 5 (*Post-BERT Era: BERT-based*) and 6 (*Post-BERT Era: T5*), in the source's own order — licensed **CC BY 4.0**: `https://creativecommons.org/licenses/by/4.0/`.
- Figures are reproduced as they appear in DL4NLP, with the attributions DL4NLP itself gives (several are credited there to the original papers or to Jay Alammar's *The Illustrated BERT*).